

Cost Optimization of Loop Water Distribution Networks

A Master's Thesis

*Submitted in partial fulfillment of the requirements for the degree of
Masters of Technology*

in

Computer Science and Engineering

by

Siddhant Singh 213050031

Supervisor:

Prof. Om Damani



Department of Computer Science and Engineering

Indian Institute of Technology

Bombay (India)

6 July 2023

Dissertation Approval

This dissertation entitled **Cost Optimization of Loop Water Distribution Networks** by **Siddhant Singh** is approved for the degree of **Master of Technology**.

Examiners:

Digital Signature
Abhiram Ranade (i95041)
29-Jun-23 05:18:40 PM

.....

Digital Signature
Ashutosh Mahajan (i12059)
30-Jun-23, 08:45:35 AM

Supervisor:

Digital Signature
Om Damani (i04119)
30-Jun-23 09:28:32 AM

Chairperson:

Digital Signature
Abhiram Ranade (i95041)
30-Jun-23 09:49:46 AM

Date: 30 June 2023

Place: IIT Bombay

Declaration

I affirm that my written submission reflects my ideas in my own words and that I have properly attributed and referenced any outside ideas or phrases that I have included. In addition, I affirm that I have followed all rules governing academic honesty and integrity and that I have not created or manipulated any idea, data, fact, or source in my work. I am aware that any breach of the aforementioned rules may result in Institute disciplinary action and may also result in penalties from the sources who were not properly cited or from whose proper permission was not obtained when required.

Date: 6 July 2023



Siddhant Singh

Roll No. 213050031

Abstract

A water distribution network is a vital infrastructure used for supplying water from the source reservoir to many end-demand nodes. These networks are the foremost infrastructure of any society and thus it is essential that their cost can be optimized as much as possible. A way to optimize their cost is through solving the optimal pipe diameter selection problem which consists of the identification of what pipe diameters are to be used at what segment of the water network.

JalTantra is a web tool that uses linear (for branched) and non-linear (for looped) programming to optimize the cost of water distribution networks by solving the optimal pipe diameter selection problem. The current implementation of JalTantra utilizes the Baron and Ocuteract solvers for looped water distribution networks. This thesis aims to explore alternative solvers that can enhance the performance of the looped JalTantra system. The research focuses not only on identifying and fine-tuning suitable solvers but also on ensuring that their solutions adhere to the constraints of the non-linear program used in the looped JalTantra System.

Additionally, this work introduces new functionalities to the loop system, such as the ability to incorporate fixed arcs with predetermined pipe diameters, allowing users to customize specific segments of the network. Furthermore, a dedicated database is implemented to store comprehensive records of solver performance, facilitating future analysis and informed decision-making regarding solver selection and commercial license procurement.

The objective of this thesis is to improve the overall performance of loop Jaltantra, both in terms of optimizing the objective cost of water distribution networks and enhancing the functionalities offered to users. The outcomes of this work have the potential to significantly benefit engineers across the country by providing them with a more efficient and cost-effective approach to designing water networks.

Table of Contents

1	Introduction	1
1.1	Objective	1
1.2	Water Distribution Networks	1
1.3	Pipe Diameter Selection Problem	1
1.4	Linear and Non-Linear Programming	2
1.5	Tools for Non-Linear Program computation	2
1.6	System architecture of Jaltantra	3
2	Methodology For Filtering and Tuning Solvers	5
3	System Architecture	7
3.1	Models and data files	7
3.2	System architecture for selecting best solver	8
4	Experimentation	11
4.1	Preliminary Experimentation for Filtering solvers	11
4.1.1	Objective	11
4.1.2	Methodology	11
4.1.3	Results	12
4.2	Detailed Experimentation on Selected Solvers	16
4.2.1	Objective	16
4.2.2	KNITRO	16
4.2.3	Baron	22
4.2.4	Octeract	25
4.2.5	Results	25
4.3	Verifying Solver Results	27
4.3.1	Objective	27
4.3.2	Constraint checker	27
4.3.3	Methodology	30
4.3.4	Observations	31
4.3.5	Fixing solvers to adhere to constraints	32
4.4	Effect of Computational Parameters on Selected Solver	36

4.4.1	Effect of Threads	36
4.4.2	Effect of Multi-Start Time	37
4.4.3	Effect of Max Iterations	38
4.4.4	Results	39
5	New Features	41
5.1	AMPL solvers integration to Jaltantra	41
5.1.1	Methodology	41
5.2	Fixed arcs implementation	42
5.2.1	Approach 1	43
5.2.2	Approach 2	44
5.2.3	Verification of fixed arc	45
5.3	Database for Storing Solver Result	46
5.3.1	Database architecture	46
5.4	Other contributions	51
6	Results	53
7	Conclusion and Future Work	57
7.1	Conclusion	57
7.2	Future work	58

List of Figures

1.1	Jaltantra System Architecture	4
2.1	Methodology Followed For Filtering and Tuning of Solvers	6
3.1	System architecture for filtering solvers	8
4.1	Preliminary experimentation methodology	12
4.2	Epanet simulation model for verification	21
4.3	Jaltantra System Architecture	30
5.1	Fixed Arc Example	43
5.2	networks.db architecture	47
a	best_solves table schema	47
b	all_solves table schema	47
5.3	Database integration architecture	50

List of Tables

3.1	Model 1 and Model 2	7
4.1	Jaltantra Objective Values	13
4.2	SNOPT preliminary experiment results	13
4.3	LOQO preliminary experiment results	14
4.4	MINLP preliminary experiment results	14
4.5	Lancelot preliminary experiment results	15
4.6	Knitro preliminary test results	15
4.7	Effect of multi-start	18
4.8	Five Minute results for different threads (Knitro - Multistart)	19
4.9	Five Minute results for different models with 48 threads	20
4.10	Flow verification	21
4.11	Head verification	21
4.12	Baron performance comparison for different threads (5 minutes)	23
4.13	Flow verification Baron	24
4.14	Head verification Baron	24
4.15	Knitro (tuned) vs Baron	25
4.16	Knitro (tuned) vs Octeract	26
4.17	Overall improvement by Knitro (potential violation of constraints)	26
4.18	Baron Constraint Violations	31
4.19	Knitro Constraint Violations	32
4.20	Knitro Performance with tuned parameters (no bug, 5 minutes)	36
4.21	Knitro Objective Costs for different Thread Configurations (5 minutes)	37
4.22	Knitro Objective Costs for Different Time Limits (4 threads)	38
4.23	Increasing maxit experiment result	39
4.24	Decreasing maxit experiment result	40
5.1	Solution obtained in Step 1	45
5.2	Solution obtained in Step 2	46
6.1	Comparison of Objective Costs between Baron and Knitro Solvers at Optimal Settings (5 minutes)	54

6.2	Comparison of Objective Costs between AlphaECP and Knitro (5 minutes)	54
6.3	Comparison of models (5 Minute solve time)	55
6.4	Comparison of Best Results of model files	55
6.5	Scenario for best result	56

Chapter 1

Introduction

1.1 Objective

The objective is to optimize the cost of building loop water distribution networks by identifying the optimal nonlinear program solvers for the Jaltantra model. Subsequently, fine-tune these selected solvers to maximize their compatibility with the Jaltantra model. Furthermore, introducing new functionalities to enhance the performance and capabilities of the loop Jaltantra system.

1.2 Water Distribution Networks

A water distribution network is an interconnection of pipes, valves, pumps, and tanks to deliver the required demand of water from the main water source to the end consumer nodes. It is a very vital infrastructure for any civilization to survive. Hence it is important that these infrastructures can be built for as little cost as possible while ensuring the demands of consumers are still satisfied.

1.3 Pipe Diameter Selection Problem

One way to reduce the cost of water distribution networks is through optimal pipe diameter selection. Pipes with larger diameters offer lesser head loss (loss in energy of water due to friction with pipe surface) but are expensive. On the other hand, smaller-diameter pipes are cheaper but lead to more head loss when compared with larger-diameter pipes. Hence the problem is to select the appropriate diameter of pipes for each link in our network such that the cost is minimal but constraints such as minimum pressure, minimum demand, and minimum flow for each consumer are still satisfied.

1.4 Linear and Non-Linear Programming

Linear and Non-linear programming both consist of an objective function that needs to be maximized/minimized while making sure some relevant constraints are not violated. If the constraint consists of only linear terms then it is a linear program otherwise it is a non-linear program. Linear programs are relatively easier to solve and reaching to global optimum is theoretically guaranteed. On the other hand, there is no such guarantee in the case of non-linear programs as there can exist many locally optimal solutions. Various commercial solvers are available in the market which can find these local optimal solutions and even global optimal solutions in a few cases, and the performance of solvers varies with different NLP models.

1.5 Tools for Non-Linear Program computation

- Jaltantra: JalTantra[1] is a freely accessible web tool that optimizes the cost of the water distribution network by solving the optimal pipe diameter selection problem. It uses linear programming[2] for branched networks and non-linear programming in the case of looped networks. While reaching a globally optimal solution in the case of linear programs is guaranteed and easy, the same thing doesn't hold in the case of non-linear programming as there can exist many locally optimum solutions. Branched water networks are typically found in rural areas. Since solving linear programs is easier, JalTantra is a great tool for assisting engineers responsible for designing these networks in rural areas. However, urban areas typically require looped water networks, and since solving nonlinear programs is harder and optimality is not guaranteed, it is very important that the solvers used by JalTantra are the best fit for the model. The currently deployed JalTantra system uses a combination of two solvers named Baron and Octeract. The work presented in this report explores the performance of other available solvers in terms of the optimality of the answer and the time taken for termination as compared to the presently deployed system.
- NEOS Server: Commercially available solvers are expensive and thus it becomes infeasible to buy them before testing. NEOS Server[3] is a platform that allows free access to more than 60 plus solvers across many optimization categories. These solvers are run by NEOS on high-performance remote machines thus acting as a one-place solution for selecting the most appropriate solver.

- EPANET: EPANET[4] is a simulation software for water distribution networks. It takes a water distribution network as input and simulates it to output various parameter values across various time periods. This work has used this simulation software to cross-check the correctness of the solutions given by the solver. Ideally, the values outputted by the solver should match those predicted by EPANET. However slight disparity is also allowed since both might be using separate libraries to solve the equation which could use different amounts of precision to give the variation in result.

1.6 System architecture of Jaltantra

In order to understand how certain features were incorporated into the current Jaltantra system, it is important to understand the system architecture of Jaltantra. Figure 1.1 shows the architecture of the current system.

The water network data provided by the user is processed in the front-end and passed to the Optimizer servlet. The servlet converts this data into respective data objects, such as Pipes (with properties like start node, end node, length, diameter, etc.), Nodes (with properties like elevation, ID, etc.), and Commercial Pipes (with properties like diameter, cost, etc.). These data objects are then passed to the Optimizer.java file.

In the Optimizer.java file, a network file is created using the received data objects. Subsequently, a shell script is invoked to call the CalculateNetworkCost.py file. This shell script passes information regarding the selected solvers, models, threads, and time constraints for solving the network file to the Python file.

Within the CalculateNetworkCost.py file, multiple ampl sessions are spawned based on the provided information, generating solution files for each solver and model combination. These solution files are then analyzed, and the path to the best solution is extracted. This path is then passed to another Python file named ExtractResult.py.

The ExtractResult.py file utilizes the solution path to generate a best solution file in a format that can be read by the Optimizer.java file. The Optimizer.java file converts this output into output objects, which are subsequently transferred to the OptimizerServlet. Finally, the results are passed to the front-end for display to the user.

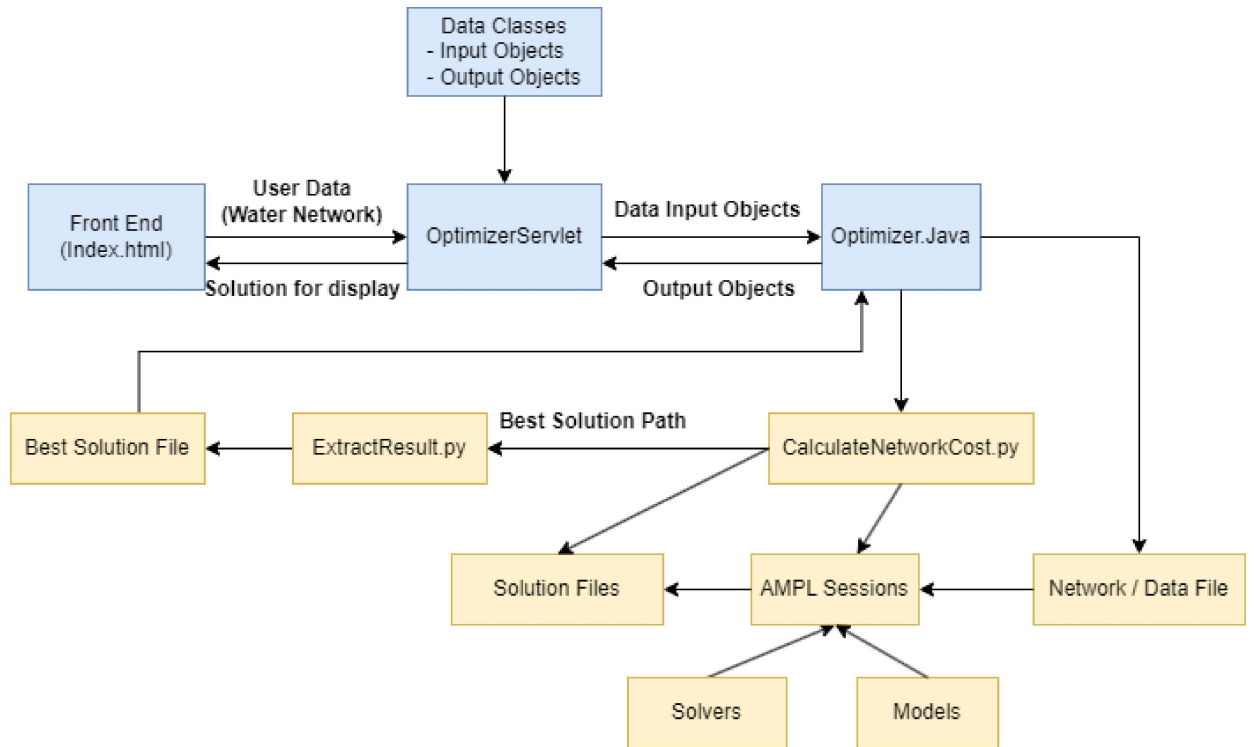


Figure 1.1: Jaltantra System Architecture

Chapter 2

Methodology For Filtering and Tuning Solvers

The initial objective of this work was to filter out certain potential solvers through some preliminary experimentation. Later, the primary objective of identifying solvers who perform better than the ones JalTantra is currently using was done by performing detailed experiments on the solvers which looked promising. The objective of detailed experiment was to tune these solvers so that they become best compatible with the Jaltantra model and also to check if the solver results adhered to the constraints. The selected candidates were then compared with each other to identify the best ones that should be integrated into the current system.

In order to compare solvers with each other, the following parameters were chosen :

- Objective function value: The most obvious parameter is how well the solver is minimizing the cost of the objective function used in JalTantra i.e cost of the water distribution network.
- Maintaining constraint integrity: Solvers often allow for a little constraint violation in order to find better results. These violations are so minuscule that often they won't have any practical significance in the real world. However, many times solvers can allow these violations to exceed the allowable thresholds. In such a case, the result given by the solver has no value, no matter how well it is minimizing the cost function. Hence, such solvers were discarded.
- Time: Another important parameter is how quick the solver is in the optimal value. A solver which can quickly arrive at an optimum is preferable to another solver which gives slightly less objective value but takes significantly more time to reach there. For this work, the following time periods were chosen:

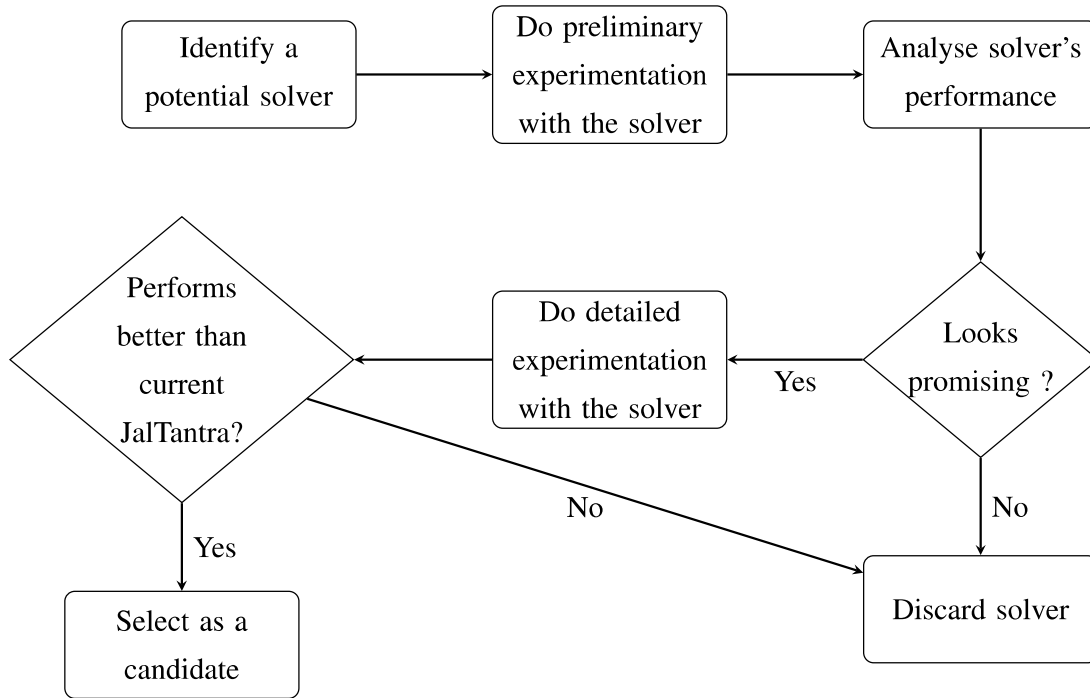


Figure 2.1: Methodology Followed For Filtering and Tuning of Solvers

– 5 Minutes – 4 Hours – 8 Hours

- Resource usage: How many CPU cores a solver requires to solve the problem efficiently is another parameter to determine how good the solver is. Ideally, if a solver can perform well even with just a single thread, it means less load on the server to solve a single job and hence more jobs can be done at the same time. Solvers performance was tested on the following number of threads:

– 1 Thread – 4 Threads – 8 Threads

Chapter 3

System Architecture

3.1 Models and data files

In the Jaltantra model, a "Model" refers to a representation of the nonlinear program formulated in a format compatible with the AMPL interface. This model encompasses an objective function that seeks to minimize the cost associated with the water distribution network. Additionally, a set of constraints is defined to ensure compliance with specific requirements such as head, flow, and pressure at various nodes within the network. The current Jaltantra loop system works with 2 different model files. These model files use a different set of constraints to capture the same problem. These models along with their respective set of constraints are shown in Table 3.1.

Model 1	Model 2
Flow Constraint	
$\sum_{i \in \gamma_j^+} q_i - \sum_{i \in \gamma_j^-} q_i = D_j$ for $j \in N$	$\sum_{i \in \gamma_j^+} (q_{1i} - q_{2i}) - \sum_{i \in \gamma_j^-} (q_{1i} - q_{2i}) = D_j$ for $j \in N$
Head Constraints	
$h_i \geq E_i + p_i$ for $i \in N$	$h_i \geq E_i + p_i$ for $i \in N$
$h_i - h_j = \sum_{k \in P} \frac{\omega \cdot l_{ak} q_a l_{aj}^{0.852}}{R_k^{1.852} d_k^{4.87}}$ for $(i, j) \in a, \forall a \in E$	$h_i - h_j = \sum_{k \in P} \frac{\omega \cdot l_{ak} (q_{1.852}^{1.852} - q_{2.852}^{1.852})}{R_k^{1.852} d_k^{4.87}}$ for $(i, j) \in a, \forall a \in E$
$h_s = E_s$	$h_s = E_s$
Pipe Length Constraint	
$\sum_{j \in P} l_{ij} = L_i$ for $i \in E$	$\sum_{j \in P} l_{ij} = L_i$ for $i \in E$

Table 3.1: Model 1 and Model 2

Some notations for reference,

1. N is the set of nodes
2. E is the set of arcs
3. P is the set of commercially available pipes
4. s is the Source node
5. $(i, j) \in a$ are the two connecting nodes of edge $a \in E$

6. γ_i^+ is the set of incoming edges for node i .
7. γ_i^- is the set of outgoing edges for node i .

To test solvers, eleven data files were used from a mixed pool of synthetically generated and real-life networks. This made up a total of 44 combinations to be run with each solver. Manually running so many combinations for one solver was tedious and demanded some automation. Apart from that, identifying what all solvers are available in the market demanded a lot of research. Even after identification, these solvers are expensive, hence using a trial license is the only feasible choice which posed further challenges in terms of what process to follow to acquire these licenses. After some research, it was identified that the NEOS server provides a platform to work around these challenges.

3.2 System architecture for selecting best solver

NEOS Server[3] is a platform that gives free-of-cost access to more than 60 commercial solvers in order to test and identify the best ones for any certain model. Thus, using NEOS server would eliminate the need for the exploration of solvers, and also there will be no need to inquire about the solver's trial license. Apart from this, all the computation would also be done on the NEOS server machines thus reducing the burden of arranging suitable computation powers. The only challenge left was to automate the process of running model-data combinations for each solver on the NEOS server.

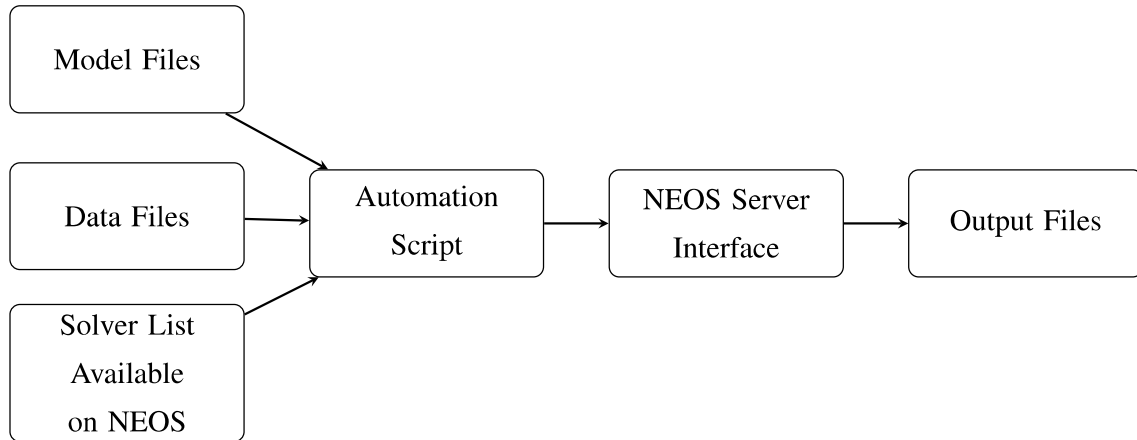


Figure 3.1: System architecture for filtering solvers

Interfaces of NEOS Server

NEOS server provides several interfaces[5] to interact with the solvers. This work demanded that the nature of the interface should allow for the automation of running model-data combinations for the reasons stated above. The available interfaces were as follows:

- **Python/XML-RPC:** The data and model files to be used with any solver are packaged as an XML file in a proper format which is available at [5]. This XML file needs to be sent to NEOS server through NEOS python client. However, this option was rejected as different solvers use different XML templates thus rendering this option unfavorable for automation. On top of that, jobs submitted through this option are limited in terms of number of constraints and hence not usable for larger data files. Thus, this option was not pursued.
- **AMPL via Krestel:** One can use locally installed AMPL to interact with NEOS server by using krestel. One only needs to select krestel as the solver and then select the actual solver in krestel-options. This option was suitable for automation as there was only one line change in the code for switching between the solvers which could be easily done through a python script. However, after running the script it was found that this option too was limited in terms of the size of the job that can be submitted. It only allowed for up to 300 variables and 300 constraints hence not useful for this work as 8 out of 11 data files violated this requirement. Hence this option was also discarded.
- **Web-interface:** The web interface of NEOS is available at [5]. This is the only interface that is not limited in terms of the size of the job that can be submitted. However, this option didn't allow any possibility of automation as one needs to submit their E-Mail id to which the result of the jobs will be submitted. After the submission of the job, a unique job id is provided on the interface, and the Email which contains the result also has the same job id mentioned in order to map jobs and results with each other. It was chosen as the interface for filtering solvers since this was the only interface that could run all 44 model-data combinations.

Limitation of NEOS server

- After selecting a solver and providing it with a data file and model file, NEOS does all the computation on its own local remote machines. The max allowable threads for a job is limited to 4, hence all the testing is restricted to just 4 cores. Performance on cores more than 4 cant be analyzed through NEOS.

- If a job time exceeds 8 hours, NEOS won't send the result by E-Mail neither it will show any result on its web interface. Hence all testing has to be restricted to a time frame of 8 hours.
- If Python or Krestel interface is used, the number of variables and constraints for a job is restricted to only 300 each, which only leaves the web interface as the option. The web interface demands the manual selection of data files and model files for each job. On the of that, it provides a job number to keep track of and the result is E-mailed, making tracking of jobs even more difficult. Hence, the overall procedure is very tedious for rigorous testing.

Chapter 4

Experimentation

4.1 Preliminary Experimentation for Filtering solvers

4.1.1 Objective

Since there were 60-plus solvers available on NEOS, it was not possible to explore every single one of them in detail due to time constraints. Hence, the initial objective was to filter out some of them on the basis of some preliminary experimentation so that the filtered ones could be explored in detail later.

4.1.2 Methodology

The initial phase of the experimentation involved assessing the performance of various solvers when applied to the model using default settings. Even if a solver's objective value exceeded the output of JalTantra, it was retained for future consideration, as it was speculated that further tuning could potentially enhance its performance. To expedite the process, the solvers were divided among three individuals for preliminary experimentation, aiming to increase efficiency.

Given that the solvers were manually tested, a criterion was established to discard solvers that failed to perform successfully with the first three data files (which were the smallest in terms of variables and constraints) across all four models. This decision was made under the assumption that if a solver proved incapable of solving the smallest data files for all models, it would likely be inadequate for larger data files as well. Consequently, only the solvers that successfully passed this initial screening process were subjected to complete experimentation across all model-data combinations.

This approach ensured a comprehensive evaluation of the solvers, as those that demonstrated early promise and passed the initial test were further examined for their efficacy and suitability across a broader range of problem instances. By systematically

eliminating solvers that were deemed incapable based on the initial assessment, the subsequent experimentation focused on a select set of solvers that exhibited the potential to deliver improved results with additional tuning.

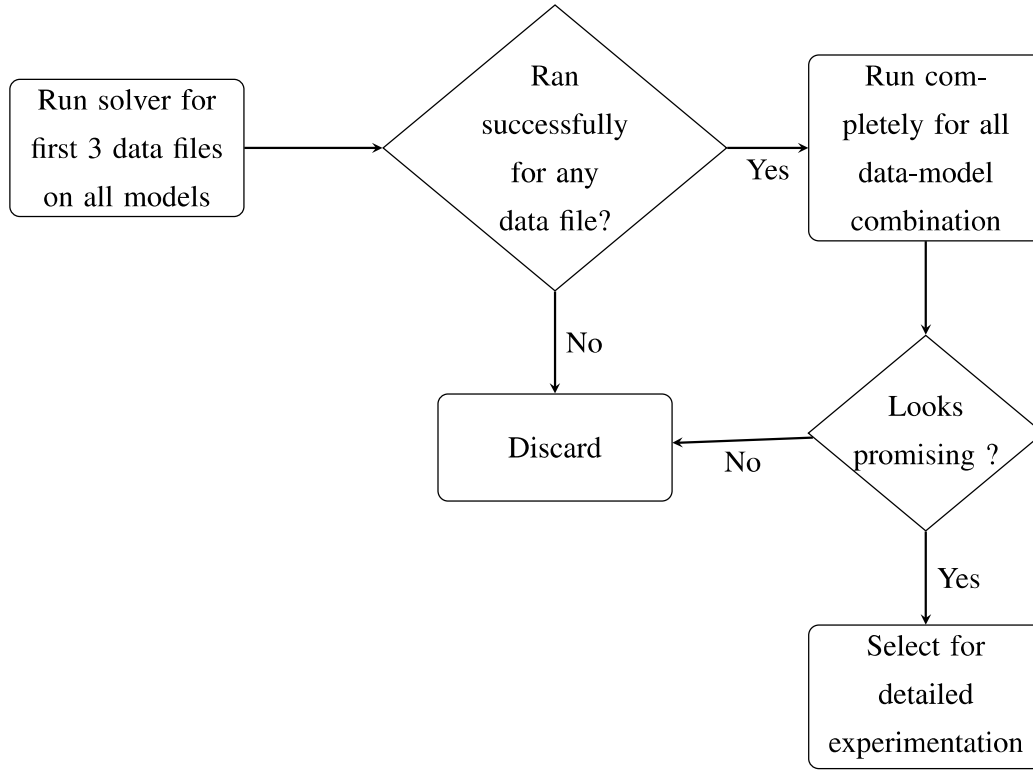


Figure 4.1: Preliminary experimentation methodology

4.1.3 Results

Here is the list of solvers that were discarded because they couldn't give objective value even for the first 3 data files on all the models :

- ASA
- COUENNE
- ICOS
- PSWARM
- BPMPD
- COPT

Solvers which survived the process were compared with current JalTantra (which uses Baron and octeract as solvers) to determine if they looked promising or not. The values outputted by current JalTantra on data files are given below:

Data file	JalTantra (Objective value)
d1	4,03,600.00
d2	60,60,000.00
d3	1,20,71,800.00
d4	1,83,85,300.00
d5	88,49,080.00
d6	35,54,290.00
d7	34,05,000.00
d8	69,20,350.00
d9	1,71,10,000.00
d10	93,29,870.00
d11	1,07,21,300.00

Table 4.1: Jaltantra Objective Values

Other solvers who survived the above process i.e were able to give output for at least one model file on any of the first three data files are discussed in detail below:

- **SNOPT:** The solver was only able to run for model 2 and model 4. The objective values outputted were much higher than the current JalTantra (table 4.1) for most of the data files. Only significant improvement was seen in data file nine. This solver was discarded from further consideration.

Solver : SNOPT				
Data File	Model 1	Model 2	Model 3	Model 4
d1	-	4,45,216.0786	-	4,16,828.04
d2	-	60,91,318.942	-	67,69,412.52
d3	-	1,25,68,277.08	-	1,30,13,016.46
d4	-	1,84,11,492.04	-	2,03,97,646.66
d5	-	95,01,025.532	-	-
d6	-	51,66,824.346	-	70,86,996.78
d7	-	74,31,206.42	-	80,75,997.20
d8	-	70,96,765.514	-	45,52,011.21
d9	-	50,65,175.896	-	13,41,969.78
d10	-	93,22,644.294	-	16,22,388.10
d11	-	1,11,45,451.39	-	1,33,89720.21

Table 4.2: SNOPT preliminary experiment results

- **LOQO:** The solver was only able to run till d3 and that too only for Model 1 and Model 2. The objective values were significantly higher than JalTantra. This solver didn't look promising and hence was discarded.

Solver : LOQO				
Data File	Model 1	Model 2	Model 3	Model 4
d1	9,54,789	9,54,789	-	-
d2	90,17,381.782	90,17,381.782	-	-
d3	2,07,70,627.08	2,07,70,627.08	-	-
d4	-	-	-	-
d5	-	-	-	-
d6	-	-	-	-
d7	-	-	-	-
d8	-	-	-	-
d9	-	-	-	-
d10	-	-	-	-
d11	-	-	-	-

Table 4.3: LOQO preliminary experiment results

- **MINLP:** The solver only ran for model-2 and model-4. Only data file nine showed significant improvement. This behavior was similar to Snopt which indicated that solvers used by JalTantra performed badly for data file 9.

Solver : MINLP				
Data File	Model 1	Model 2	Model 3	Model 4
d1	-	4,25,552.8163	-	4,24,099.791
d2	-	62,68,525.976	-	62,93,428.16
d3	-	1,71,59,803.12	-	1,29,79,158.97
d4	-	2,88,84,605.01	-	2,57,74,620.7
d5	-	90,18,662.887	-	-
d6	-	37,43,572.354	-	38,01,651.315
d7	-	33,95,916.94	-	34,03,157.822
d8	-	70,34,584.491	-	70,81,200.633
d9	-	89,43,408.54	-	-
d10	-	1,01,72,415.43	-	1,01,72,415.43
d11	-	1,11,61,327.45	-	1,11,61,327.45

Table 4.4: MINLP preliminary experiment results

- **Lancelot:** The solver ran completely for model-1 and model-2 and incompletely for model-3 and model-4. There was a significant improvement in objective value for all data files. This solver was selected for a detailed examination.

Solver : Lancelot				
Data File	Model 1	Model 2	Model 3	Model 4
d1	1,17,205.816	1,18,853.7452	4,26,147.338	2,47,117.765
d2	60,62,283.338	60,59,624.552	64,38,749.286	60,31,762.177
d3	1,20,85,751.79	1,20,77,160.1	1,26,59,132.97	1,20,71,189.37
d4	1,83,92,196.84	1,84,03,831.58	1,96,24,412.86	1,84,19,309.45
d5	93,45,750.854	88,84,748.237	-	-
d6	37,07,681.868	20,34,534.213	-	9,15,124.5204
d7	38,42,033.76	13,75,377.436	-	7,28,226.5185
d8	62,46,696.698	6,28,911.3302	-	-
d9	90,85,273.085	20,06,355.989	-	-
d10	93,73,748.627	17,40,024.924	-	-
d11	99,39,866.499	22,59,117.177	-	-

Table 4.5: Lancelot preliminary experiment results

- **KNITRO:** The solver ran completely for model-1 and model-2. There was no improvement in objective values across any data file but values were close to what JalTantra was already outputting.

Solver : KNITRO				
Data File	Model 1	Model 2	Model 3	Model 4
d1	4,03,603.0367	61,43,490.542	4,39,640.87	6,62,587.58
d2	61,46,670.49	90,17,381.782	73,39,222.51	68,21,973.08
d3	1,21,98,858.29	1,22,80,754.66	1,54,70,127.4	1,21,98,518.63
d4	1,91,15,262.12	1,85,14,218.63	2,71,95,282.65	-
d5	92,72,181.604	92,08,761.98	-	-
d6	35,91,424.233	69,99,737.92	-	-
d7	33,93,129.285	34,03,157.82	-	-
d8	95,95,129.278	95,95,129.28	-	-
d9	90,69,576.741	89,77,620.74	-	-
d10	95,50,155.987	94,68,239.09	-	-
d11	1,09,49,904.44	1,64,62,299.09	-	-

Table 4.6: Knitro preliminary test results

Conclusion

On the basis of the comparison of the performance of each solver with table 4.1, the following actions were taken:

- LOQO was straight away rejected as it wasn't able to run more than 3 data files.

- Only KNITRO and LANCELOT were chosen for detailed experimentation as their results were comparable to the JalTantra.
- SNOPT and MINLP were rejected as their results were a little off from what JalTantra was already giving.

4.2 Detailed Experimentation on Selected Solvers

4.2.1 Objective

Based on initial experimentation, two solvers, namely Lancelot and Knitro, demonstrated promising results comparable to, or even surpassing, the performance of the existing solver used by JalTantra. However, in phase-1 of this work, it came to notice that Lancelot was not adhering to the constraints of the Jaltantra model and hence was discarded from further experimentation. Consequently, only Knitro was selected for further comprehensive exploration. Given that the current JalTantra utilized Baron and Octeract as its solver, in-depth experiments were also conducted on both of these to investigate the potential for enhancing their performance. The objective of the detailed experiment was to tune these solvers to be the most compatible with the Jaltantra model and also to check whether these solvers adhered to the constraints. The findings and analyses of these experiments are elaborated upon in the subsequent sections.

4.2.2 KNITRO

Knitro in preliminary experimentation was giving higher objective cost than JalTantra but the results were close and demanded further exploration of the solver. The documentation with all options of Knitro is available at [6].

Algorithms and related options

Knitro provides 5 different algorithm options. In case this option is not used, Knitro automatically selects the best ones according to the problem. However, manually testing each algorithm is recommended. The available algorithm options present in Knitro are:

- Interior Point Method / Direct(algorithm = 1): These methods work by replacing the non-linear programming with many barrier subproblems which can be controlled by a barrier parameter. If negative curvature is detected it will automatically switch to algorithm 2. More information about this algorithm is available at [6].

- Interior Point Method / CG (algorithm = 2): This method is best if Hessian is too dense or large. It is suitable for larger problems as it avoids factorizing the hessian matrix. The option `bar_murule` is used for specifying what update rule to use with these interior point methods. Another option `linsolver` specifies which linear solver to use when solving the Newton step. However, none of them seemed to give an advantage over the other.
 - Active set Method (algorithm = 3): This option is suitable for small to medium-scale problems and is not recommended over algorithms 1 and 2 if the problem size is large. It can work better if the user has some good estimate about the initial solution.
 - Sequential Quadratic Programming (SQP) (algorithm = 4): It is also an active set method, it solves a series of quadratic subproblems to solve the bigger problem. It is only suitable for small to medium-scale problems.
 - Multi Algorithm (Algorithm = 5): This option will make Knitro run all the above-mentioned methods on the problem either parallelly or sequentially. It then reports the performance of each method on the problem. This was the option that was used to check if any particular algorithm was performing better.
- It was found that no algorithm was consistently performing better than others on all data files. Rather, letting Knitro decide which algorithm to use by using the default algorithm option was the best choice.

Knitro Auto-tuner

Knitro provides an option for auto-tuning which can be used by setting the option `tuner = 1`. This option runs the problem with various combinations of options available on Knitro and suggests some custom option settings to try. However, no suggestions recommended by auto tuner seemed to work for the JalTantra model.

Multi-start

Non-linear problems are non-convex in nature due to which there can exist multiple local optimum solutions. The default nature of Knitro is to return the first local optimum solution. With the multi-start option, the user can allow search for local or global optimums from multiple start points and Knitro will return the best-found local optimum or the global optimum if found. Users can specify the number of start points say "m" and

Effect of different Multistarts						
Data File	20 Multistart		10 Multistart		5 Multistart	
-	Obj. Value	Time(s)	Obj. Value	Time (s)	Obj. Value	Time(s)
d1	403,498.96	2.02	403,603.04	0.65	403,603.04	0.39
d2	6,059,622.00	15.84	6,061,478.00	5.07	6,070,900.30	1.21
d3	12,071,816.86	35.10	12,094,342.96	13.55	12,198,858.29	4.12
d4	18,384,232.44	18.13	18,589,813.97	12.73	18,595,037.22	13.80
d5	8,838,146.70	58.03	8,879,367.00	10.43	8,955,253.11	2.45
d6	3,553,170.74	83.41	3,54,279.00	52.13	3,578,127.90	25.20
d7	3,388,105.84	80.16	3,393,129.29	42.71	3,511,971.88	13.48
d8	5,820,920.08	1,348.60	5,849,089.09	920.69	5,914,873.94	245.97
d9	8,686,817.00	434.33	8,884,526.04	140.31	8,983,065.56	70.68
d10	9,329,079.01	259.12	9,341,735.71	190.53	9,341,735.71	87.72
d11	10,715,139.97	382.03	10,715,139.97	150.49	10,840,996.49	92.77

Table 4.7: Effect of multi-start

then Knitro will estimate good "m" starting points to start the search. Multi-start can be done in two ways:

- Serially: Each start point is solved one after the other by using all the available threads in one iteration.
- Parallely: Start points are solved parallely by dividing the threads between different start points.

Table 4.7 depicts the results of using various multi-start points with 4 threads on Model-1. Model-1 was chosen as it was taking least time to give output during preliminary experimentation. After comparing table 4.6 and 4.7, it is evident that using multi-start has drastically improved the solver's performance. From table 4.7 it is also evident that using higher multi-start values gives better results but takes up more computation time. Keeping the above conclusion in mind, it was decided to experiment with multi-start with different amounts of CPU cores and let it run as many multi-starts as possible within 5 minutes without any restrictions.

Impact of multi-cores on multi-start

Since a greater number of multi-starts was giving better objective value, it was decided not to limit the number of multi-starts, but rather let as many multi-start iterations run as possible in 5 minutes. This number would depend on how many cores there are in the

machine as the greater the number of cores, the greater will be the number of multi-start instances that finish in 5 minutes. Since NEOS didn't allow more than 4 cores, a trial license was acquired for Knitro for its further testing. Following is the result given by different amounts of threads on Multi-Start for Model-1 within 5 minutes:

Impact of threads on multistart				
Data File	1 Thread	4 Thread	16 Thread	48 threads
d1	3,99,198.02	3,17,190.94	3,09,808.67	3,09,808.67
d2	60,59,621.75	60,59,621.75	60,59,621.69	60,59,621.69
d3	1,20,71,816.84	1,20,71,816.45	1,20,71,816.45	1,20,71,815.85
d4	1,83,92,190.02	1,83,85,478.29	1,83,85,478.29	1,83,85,478.29
d5	88,24,825.39	88,16,379.72	88,16,379.41	88,16,379.41
d6	35,56,311.16	35,55,793.84	35,53,498.59	35,53,498.59
d7	33,90,385.49	33,88,106.05	33,88,105.86	33,88,105.86
d8	59,14,873.94	58,45,993.62	58,41,680.86	58,41,680.86
d9	88,71,465.94	87,05,739.65	87,05,739.65	88,30,486.68
d10	93,41,735.71	93,32,716.51	93,32,716.51	93,32,716.51
d11	1,07,22,463.15	1,07,22,463.15	1,06,77,204.24	1,06,97,104.71

Table 4.8: Five Minute results for different threads (Knitro - Multistart)

Apart from d1 which was the smallest data file, increasing the number of threads didn't seem to have a significant impact on the improvement of results.

Impact of model on Multi-Start

Table 4.8 only shows the impact of threads on model-1. Other models were also explored with multi-start to see if there is any improvement in performance. This experiment was conducted with 48 threads because if a model doesn't work with max allowable threads, then it is guaranteed that it won't run with a lesser number of threads as well. In that case, that particular model can be eliminated from consideration.

Table 4.9 shows the result of this experiment. It is seen that Model-2 was able to give results on the first 8 data files only while Model-3 and Model-4 gave results till the 4th data file. Hence these three models were removed from consideration and only model 1 emerged as the potential choice.

Impact of model on multi-start				
Data File	Model 1	Model 2	Model 3	Model 4
d1	3,09,808.67	3,40,385.13	3,87,756.33	3,22,405.58
d2	60,59,621.69	60,59,621.92	60,59,622.02	60,59,621.99
d3	1,20,71,815.85	1,20,71,816.86	1,24,23,624.35	1,20,74,488.68
d4	1,83,85,478.29	1,84,32,356.06	1,90,72,726.83	2,08,24,729.71
d5	88,16,379.41	88,20,364.00	-	-
d6	35,53,498.59	35,54,957.64	-	-
d7	33,88,105.86	33,97,487.69	-	-
d8	58,41,680.86	69,20,355.44	-	-
d9	88,30,486.68	-	-	-
d10	93,32,716.51	-	-	-
d11	1,06,97,104.71	-	-	-

Table 4.9: Five Minute results for different models with 48 threads

Knitro result verification

Just like Lancelot, file d1 was selected for verification of Knitro as it was the smallest file in terms of number of nodes and pipes. Figure ?? shows the structure of the network d1. The basic constraints of flow, minimum pressure, and pipe length were satisfied by the solution given by Knitro. For further verification, a simulation model of d1 was built on Epanet in accordance with the solution given by Knitro to cross-check if the flow and head predicted by Knitro matched with that of Epanet. Figure 4.2 shows the simulation model built on EPANET for verification. Tabel 4.10 and table 4.11 shows the comparison between the solution given by Knitro and Epanet simulation values for flow and head respectively. The difference in flow value in table 4.10 is negligible which indicates the solution is correct in terms of flow. However, table 4.11 indicates that the difference in head values although less, but is not ignorable and demands further investigation on bigger data files before selecting Knitro as the solver to be integrated into JalTantra. An initial hypothesis was that Epanet may be using different libraries for calculations hence justifying the slight difference in calculated values between them, however, it was later found that results given by Baron are hundred percent similar to that of Epanet (table 4.13 and table 4.14) thus highlighting that there might be some issue in Knitro result's demanding further investigation.

Link	Knitro Flow	Epanet Flow	Difference
1-2	311.109	311.11	0.001
2-3	27.9689	28	0.0311
2-4	255.362	255.34	-0.022
3-5	0.191934	0.22	0.028
4-5	74.6161	74.59	-0.0261
4-6	147.413	147.41	-0.003
6-7	55.7469	55.75	0.0031
7-5	0.191934	0.19	-0.0019

Table 4.10: Flow verification

Node	Knitro Head	Epanet Head	Difference
1	210	210	0
2	206.461	208.01	-1.549
3	190	191.54	-1.54
4	200.463	199.7	0.763
5	180	179.28	0.72
6	195	193.94	1.06
7	190	188.95	1.05

Table 4.11: Head verification

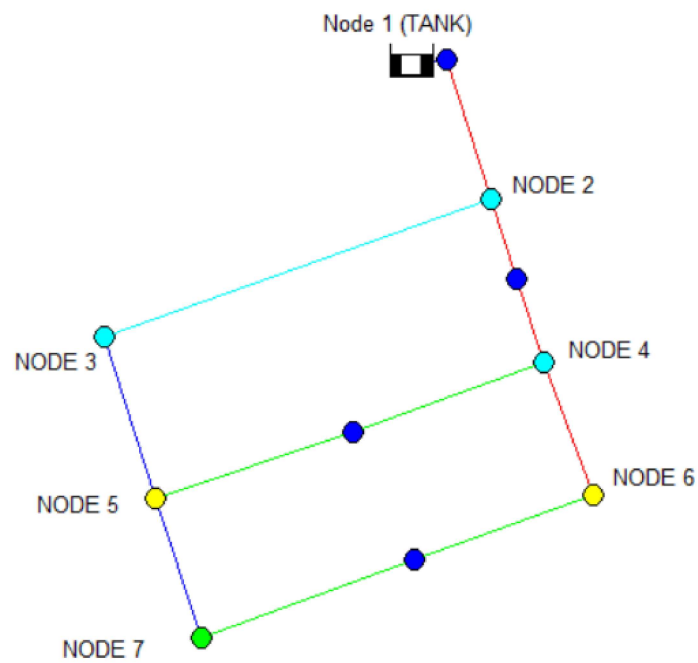


Figure 4.2: Epanet simulation model for verification

4.2.3 Baron

Baron, which is currently used as a solver in the Jaltantra system, has primarily served as the benchmark against which the results of other solvers have been compared. However, the potential for enhancing Baron's own performance remained a possibility to be explored. The settings and configurations available for Baron are listed in [9]. Prior to any tuning of the solvers, Baron demonstrated superior performance for 9 out of the 11 available data files, while giving unsatisfactory results for the remaining two. The subsequent work primarily focused on investigating the feasibility of fine-tuning Baron to achieve satisfactory results for all eleven data files.

LP solver

The option referred to as "lpsolver" in the Baron option settings dictates the choice of sub-solver utilized by Baron for solving linear constraints. Its impact on performance can be significant, particularly when integer variables are present in the problem. However, since all variables in the Jaltantra model are continuous, the expectation was that this option would have a limited effect. . To assess and compare the performance of the available options, each option was executed for a duration of 5 minutes, utilizing 4 threads, specifically for the m1 model file. The available options are as follows :

- CBC
- Cplex
- Xpress

The last 2 sub solvers required a license while the first is free. We had licenses of all 3 as part of AMPL purchase. However, no improvement was observed over the default setting when any of these lp solvers was used.

NLP solver

The list of local nonlinear solvers that Baron is allowed to use is specified by the "nlp-sol" option in the Baron solver settings. The matching values (mod 16) of the available solvers are added up to get the option value. The options that are accessible and their corresponding values are as follows:

- 1 : IPOPT (builtin)
- 2 : FilterSD (builtin)
- 4 : FilterSQP (builtin)
- 8 : lsolver (if lsolver=... is specified)

- Default 0 : Allow all

Hence various possible value combinations are:

- 3 (1 + 2): Use IPOPT and FilterSD (Add table)
- 6 (2 + 4): Use FilterSD and FilterSQP (Add Table)
- 7 (1 + 2 + 4): Use IPOPTand FilterSD and FilterSQP (Add table)

It was observed that none of the above parameter values gave any additional benefit over the default setting.

Threads

The "threads" option in the Baron solver settings determines the maximum number of threads that can be utilized when solving problems with integer variables. By default, the value is set to 1, meaning that only a single thread is used for computation. When solving optimization problems involving integer variables, the solver employs a branch-and-bound algorithm, which involves exploring different branches of the problem's solution space. The use of multiple threads can significantly enhance the efficiency and speed of the solving process by allowing for parallel computation. The difference in the performance of Baron for 1, 4, and 8 threads is shown in table 4.12. Note that blank entries indicate that Baron was not able to give satisfactory result within the specified time frame.

Data File	1 Thread	4 Threads	8 Threads
d1	4.04E+05	4.04E+05	4.04E+05
d2	6.06E+06	6.06E+06	6.06E+06
d3	1.21E+07	1.21E+07	1.21E+07
d4	1.84E+07	1.84E+07	1.84E+07
d5	-	-	-
d6	3.55E+06	3.55E+06	3.55E+06
d7	-	-	-
d8	6.92E+06	6.92E+06	6.92E+06
d9	-	-	-
d10	9.44E+06	9.32E+06	9.32E+06
d11	1.07E+07	1.07E+07	1.07E+07

Table 4.12: Baron performance comparison for different threads (5 minutes)

It can be inferred from the findings shown in Table 4.12 that there is an insignificant difference in Baron's performance when using various numbers of threads. The changes

in objective values between 1, 4, and 8 threads are hardly noticeable. As a result, it can be concluded that if the Baron solver can solve the pipe diameter selection problem for any water network then one thread is sufficient. On the other hand, if it can't solve a network, increasing the number of threads wouldn't help.

Baron result verification

To verify the accuracy of Baron's results, the d1 dataset was chosen due to its smaller network size and ease of result verification. In order to assess the compliance of the obtained solution with the specified constraints, an Epanet simulation (figure 4.2) was constructed, and the output provided by Baron was inputted into it. The calculated values of head and flow from the Epanet simulation were then compared with the corresponding values indicated by Baron. This comparison served as a means to determine if the constraints were appropriately upheld by the solution obtained from Baron.

Link	Baron Flow	Epanet Flow	Difference
1-2	311.109	311.11	0.001
2-3	27.9689	28	0.0311
2-4	255.362	255.34	-0.022
3-5	0.191934	0.22	0.028
4-5	74.6161	74.59	-0.0261
4-6	147.413	147.41	-0.003
6-7	55.7469	55.75	0.0031
7-5	0.191934	0.19	-0.0019

Table 4.13: Flow verification Baron

Node	Baron Head	Epanet Head	Difference
1	210	210	0
2	206.461	206.04	0.061
3	190	190.04	0
4	200.463	200.47	0.07
5	180	179.98	0.02
6	195	195.0	0
7	190	190	0

Table 4.14: Head verification Baron

It is clear from table 4.13 and table 4.14 that Baron is respecting the constraints and the results provided are correct. However, this shows that further investigation is

warranted regarding the results obtained from Knitro, as indicated by the observations presented in Table 4.10 and Table 4.11. These tables suggest that the performance and adherence to constraints by Knitro may require additional scrutiny and analysis.

4.2.4 Octeract

In evaluating the performance of Octeract as an optimization solver for large networks in comparison to Baron and Knitro, it became evident that Octeract's performance was significantly poorer. Table 4.16 presents the performance comparison, clearly illustrating Octeract's limitations when dealing with large networks. Unfortunately, during the time of writing this report, no available documentation could be found on the internet that provided insights into the available options for tuning Octeract. Additionally, the previously used link to explore Octeract options is no longer accessible [11], and the official website [10] does not provide any relevant documentation. The Octeract license that was used previously for testing its performance expired during the research period. Keeping the above-mentioned points in mind it was decided to discard Octeract from future considerations and only focus on Knitro and Baron as potential ampl solvers for Jaltantra.

4.2.5 Results

After complete tuning of Knitro, table 4.15 and 4.16 indicate how much improvement in cost function Knitro offers over Baron and Octeract (current Jaltantra solvers). These values are taken after letting the solvers run for five minutes.

Data file	Knitro	Baron	Improvement %
d1	3,09,808.67	4,03,607.00	23.24%
d2	60,59,621.69	60,64,800.00	0.09%
d3	1,20,71,815.85	1,20,71,800.00	0.00%
d4	1,83,85,478.29	1,83,85,300.00	0.00%
d5	88,16,379.41	88,49,080.00	0.37%
d6	35,53,498.59	35,54,290.00	0.02%
d7	33,88,105.86	2,10,99,400.00	83.94%
d8	58,41,680.86	69,20,350.00	15.59%
d9	88,30,486.68	3,23,23,000.00	73.07%
d10	93,32,716.51	93,29,870.00	-0.03%
d11	1,06,97,104.71	1,07,21,300.00	0.41%

Table 4.15: Knitro (tuned) vs Baron

Data file	Knitro	Octeract	Improvement %
d1	3,09,808.67	4,03,600.00	23.24%
d2	60,59,621.69	60,60,000.00	0.01%
d3	1,20,71,815.85	1,22,80,000.00	1.70%
d4	1,83,85,478.29	1,83,90,000.00	0.02%
d5	88,16,379.41	90,20,000.00	0.26%
d6	35,53,498.59	35,55,000.00	0.04%
d7	33,88,105.86	34,05,000.00	0.50%
d8	58,41,680.86	70,17,000.00	16.75%
d9	88,30,486.68	1,71,10,000.00	49.12%
d10	93,32,716.51	1,75,60,000.00	46.85%
d11	1,06,97,104.71	1,83,80,000.00	41.91%

Table 4.16: Knitro (tuned) vs Octeract

Table 4.17 shows the overall improvement over the current JalTantra system which uses a combination of both solvers.

Data file	Improvement %
d1	23.24%
d2	0.01%
d3	0%
d4	0.37%
d5	0.02%
d6	0.04%
d7	0.5%
d8	15.59%
d9	49.12%
d10	-0.03%
d11	0.41%

Table 4.17: Overall improvement by Knitro (potential violation of constraints)

Conclusion

After conducting detailed experiments on selected solvers, it can be concluded that only Knitro and Baron are suitable for use with the loop Jaltantra system. After parameter tuning, the best set of parameters identified for Knitro is given below :

- threads = 4
- ms_maxsolves = 100000
- ms_enable = 1
- ms_maxtime_real = 300

- maxit = 10000

However, no tuning could be done for Baron and it is used at its default setting. One important thing to note here is that the result validation of Knitro is still not conclusive and hence requires further investigation. Baron however has shown adherence to the constraints. Thus, there was a need to conduct separate experiments to check whether solvers adhered to constraints or not.

4.3 Verifying Solver Results

4.3.1 Objective

The reliability of solver outputs was brought into question by the observations in Tables 4.10 and 4.11 and the manual calculations done for Lancelot, which indicated that the results could not be completely trusted. Thus, it became very important to verify whether the obtained solutions respected the specified constraints. Manual verification for each data file was impractical due to the limitation of time and effort involved. Therefore, a programmatic approach was taken to address this issue.

4.3.2 Constraint checker

To facilitate constraint verification, a dedicated constraint checker was integrated into the Jaltantra system. This module received the results generated by the solver and performed a thorough evaluation of the differences between the left-hand and right-hand sides of each constraint. Given that non-linear solvers are not entirely accurate, it was anticipated that minor deviations would exist. Consequently, a threshold for violation was established for each constraint, considering its relative importance.

If the constraint checker detected any inconsistencies indicating a violation of the constraints, the corresponding results were discarded, and an error message was displayed to alert the user. This automated verification process ensured that only valid and constraint-compliant solutions were considered, enhancing the reliability and accuracy of the Jaltantra system.

The Jaltantra system accomplished an additional layer of validation and quality assurance by including this constraint checker, reducing the possibility of incorrect or non-compliant solutions. The programmatic method made it possible to efficiently and consistently verify constraints across any given water distribution network, improving the overall integrity and dependability of the optimization results.

Constraints and allowed tolerance

The Jaltantra model had 5 constraints that the solution of the solvers must adhere to. These are explained below :

- **Flow constraint:** At any given node, the sum of inflows should be equal to the sum of outflow + demand at that node. This is expressed mathematically as:

$$\forall i \in \text{Nodes}, \quad \sum_{\forall (i,j) \in \text{Arcs}} q[i, j] - \sum_{\forall (j,i) \in \text{Arcs}} q[j, i] = D[i] \quad (4.1)$$

The constraint checker calculates the left-hand side and right-hand side of the equation and checks if:

$$\text{abs(LHS - RHS)} > \text{ALLOWED_FLOW_TOLERANCE}$$

The **ALLOWED_FLOW_TOLERANCE** variable has been set to 10^{-4} . In case the above condition holds true, it discards the solution and throws an error to the system indicating flow constraint has been violated.

- **Minimum pressure constraint:** At any given node, the head calculated at that node by the solver should be equal to the sum of the elevation of the node and the minimum pressure required at that node. This is expressed mathematically as:

$$\forall i \in \text{Nodes} : \quad H[i] \geq E[i] + P[i] \quad (4.2)$$

The constraint checker calculates the left-hand side and right-hand side of the equation and checks if:

$$\text{LHS} - \text{RHS} < 0$$

In case the above condition holds true, it discards the solution and throws an error to the system indicating pressure constraint has been violated.

- **Head-Loss constraint:** At any given arc, the difference of the source node head and destination node head should be equal to headloss calculated using the Hazen William formula [7] between those nodes. This is expressed mathematically as:

$$\forall i, j \in \text{Arcs}, : \quad H[i] - H[j] = \text{Headloss calculated using Hazen William's equation [7]} \quad (4.3)$$

The constraint checker calculates the left-hand side and right-hand side of the equation and checks if:

$$\text{abs}(\text{LHS} - \text{RHS}) > \text{ALLOWED_HEADLOSS_TOLERANCE}$$

The **ALLOWED_HEADLOSS_TOLERANCE** variable has been set to 10^{-4} . In case the above condition holds true, it discards the solution and throws an error to the system indicating the head-loss constraint has been violated.

- **Pipe length constraint:** At any given arc, the sum of commercial pipes used at that arc should be equal to the total length of the arc. This is expressed mathematically as:

$$\forall i, j \in \text{Arcs}, \sum_{k \in \text{Pipes}} l[i, j, k] = L[i, j] \quad (4.4)$$

The constraint checker calculates the left-hand side and right-hand side of the equation and checks if:

$$\text{abs}(\text{LHS} - \text{RHS}) > \text{ALLOWED_PIPELENGTH_TOLERANCE}$$

The **ALLOWED_PIPELENGTH_TOLERANCE** variable has been set to 10^{-3} . In case the above condition holds true, it discards the solution and throws an error to the system indicating the pipe length constraint has been violated.

- **Source head constraint:** This constraint states that the calculated head of the source node by the solver should be equal to its elevation, i.e:

$$h[\text{Source}] = E[\text{Source}] \quad (4.5)$$

The constraint checker calculates the left-hand side and right-hand side of the equation and checks if:

$$\text{LHS} = \text{RHS}$$

In case the above condition holds true, it discards the solution and throws an error to the system indicating the source head constraint has been violated.

Constraint checker architecture

In the Jaltantra system architecture (Figure 1.1), an additional layer of constraint checking was implemented to ensure the integrity of the generated solutions. The `extractResult` file was identified as the suitable location to incorporate this constraint-checking functionality. Figure 4.3 shows the architecture of the constraint checker.

Before generating the solution file for Optimizer.java, the solution passed through a code module that systematically checked each constraint. At each stage, the code verified if the solution adhered to the specified constraints. If any constraint violation was detected, the code returned an exit code of -1, indicating that the solution was invalid and should be discarded.

This message was then propagated back to the CalculateNetworkCost file, which triggered the appropriate actions to handle the incorrect solution. The message was further propagated all the way to the front-end interface, where the user was notified about the constraint violation error.

On the other hand, if the solution successfully passed all stages of constraint checking without any violations, it proceeded with the normal behavior of generating the solution file for Optimizer.java. This ensured that only valid and constraint-compliant solutions were further processed and utilized.

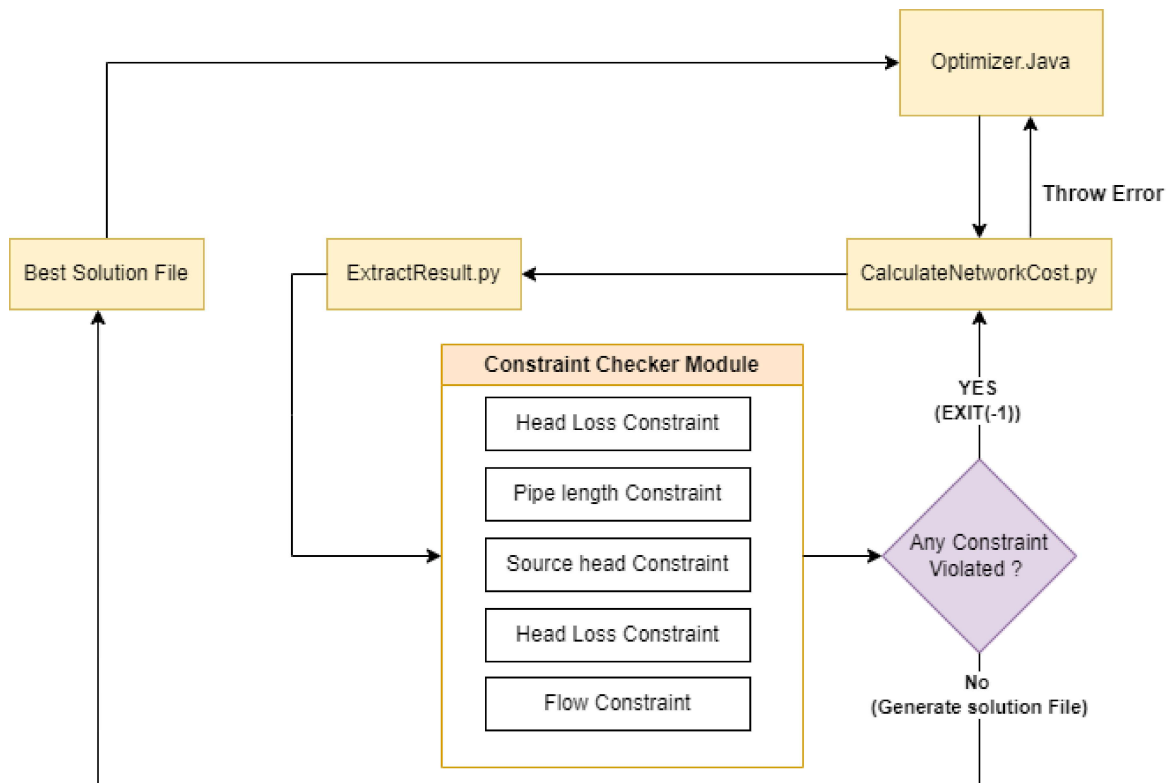


Figure 4.3: Jaltantra System Architecture

4.3.3 Methodology

In order to assess the accuracy of the results produced by Baron and Knitro solvers, both of these solvers were incorporated into the Jaltantra system. The system was then

tested using all 11 data files, and the solutions generated by each solver were subjected to constraint checking. If the system returned a solution without any errors, it indicated that the solution had successfully passed the constraint-checking phase. However, if the system encountered an error, it indicated that certain constraints had been violated. The specific constraint that was violated could be identified by examining the corresponding log file.

4.3.4 Observations

Constraint violation by Baron

Surprisingly, during the evaluation process, it was discovered that the solutions generated by the Baron solver were found to violate constraints in 3 out of 11 data files. This outcome was unexpected, especially considering the earlier observations presented in Table 4.13 and 4.14. The specific violations and corresponding data files have been documented in Table 4.18. Note that "Yes" in any entry indicates that the respective constraint is violated for that data file. The unexpected behavior observed in the validation of Baron's

Data File	Head Loss	Flow	Pipe Length	Source Head	Minimum Pressure
d1	Yes	Yes	Yes	No	No
d2	Yes	Yes	Yes	No	No
d3	No	No	No	No	No
d4	No	No	No	No	No
d5	No	No	No	No	No
d6	Yes	Yes	Yes	Yes	Yes
d7	No	No	No	No	No
d8	No	No	No	No	No
d9	No	No	No	No	No
d10	No	No	No	No	No
d11	No	No	No	No	No

Table 4.18: Baron Constraint Violations

solution for data file d1, where the solution appeared to adhere to the flow and headloss constraints in manual simulations (4.13 & 4.14) but was flagged as violating the constraints in Table 4.18, raised the need for a more thorough investigation. This discrepancy between the manual observations and the constraint validation results warranted a detailed analysis to identify the underlying factors causing this inconsistency.

Constraint violation by Knitro

Contrary to our initial belief that Knitro solutions consistently violated constraints, further analysis revealed that it only violated 2 constraints in 2 out of the 11 data files (d1 and d8). This performance is even better than that of Baron. The observations are reported in Table 4.19.

Data File	Head Loss	Flow	Pipe Length	Source Head	Minimum Pressure
d1	Yes	No	No	No	Yes
d2	No	No	No	No	No
d3	No	No	No	No	No
d4	No	No	No	No	No
d5	No	No	No	No	No
d6	No	No	No	No	No
d7	No	No	No	No	No
d8	Yes	No	No	No	Yes
d9	No	No	No	No	No
d10	No	No	No	No	No
d11	No	No	No	No	No

Table 4.19: Knitro Constraint Violations

4.3.5 Fixing solvers to adhere to constraints

Fixing Baron

In the detailed experimentation phase, it was found that Baron did not appear to violate any constraints when the results were examined through epanet simulation. However, the constraint checker reported violations for Baron in three specific data files. This inconsistency prompted a deeper analysis to uncover the underlying cause. It was hypothesized that the discrepancy could be attributed to the integration of Baron into the Jaltantra system, rather than an inherent issue with Baron itself. This was because the first set of solutions that didn't violate the constraint were outputted through the NEOS server while the ones which did violate the constraint were outputted through the Jaltantra system. Thus the difference between the two settings was noted down to narrow down the main culprit. The settings of Baron used were exactly the same across both.

Further analysis revealed that the constraint violations occurred specifically for smaller data files. Upon closer examination, it was discovered that the issue stemmed from the premature termination of Baron. When Baron found a solution, a "SIGINT" signal or the "Ctrl + C" command was sent to terminate the process and extract the results. However,

on smaller data files, this signal was being sent even before Baron terminated on its own which caused an abnormal behavior of Baron reporting the current point rather than the previous best solution found.

To address this problem, it was crucial to ensure that Baron terminated properly before any "SIGINT" signals were sent. Once this adjustment was made, Baron consistently produced solutions that respected the constraints for all data files, without any impact on the overall cost. To achieve the desired behavior, a modification was made to the settings of Baron in order to control its execution time. Specifically, the time limit for Baron was set to 5 minutes, ensuring that it would run for a fixed duration. Additionally, a slight delay of 10 seconds was introduced before sending the SIGINT signal to the Baron process.

Fixing Knitro

- **Knitro feasibility options:** In contrast to Baron, Knitro exhibited constraint violations in both the NEOS and Jaltantra systems, suggesting an underlying issue with the solver itself. These violations were observed in only 2 out of the 11 data files, prompting a focused investigation into the solver's behavior specifically on these files.

Upon closer examination, a key distinction was identified between the files where constraint violations occurred and those where no violations were observed. This distinction centered around the feasibility error values reported in the solution files by Knitro. According to the Knitro manual [6], these feasibility error values indicate the extent to which Knitro deviates from satisfying the constraints. It was observed that in the files where constraint violations occurred, the feasibility error values surpassed the threshold of 10^{-5} , whereas in the remaining files, the values were significantly smaller, typically below 10^{-7} .

In order to address the issue of constraint violations in Knitro, the Knitro options manual [6] was consulted for potential fixes. Four relevant options that pertained to how Knitro determines the feasibility of a solution were identified: `feastol`, `feastol_abs`, `opttol`, and `opttol_abs`. These options are further explained below:

- `feastol`: This option specifies the final relative stopping tolerance for the feasibility error. By setting a smaller value for `feastol`, the solution can achieve a higher degree of accuracy in terms of feasibility. The default value for `feastol` is $1.0e-6$.

- `feastol_abs`: It determines the final absolute stopping tolerance for the feasibility error. A smaller value of `feastol_abs` leads to a higher level of accuracy in the solution’s feasibility. The default value for `feastol_abs` is $1.0e-3$.
- `opttol`: This option sets the final relative stopping tolerance for the optimality error. Reducing the value of `opttol` enhances the solution’s accuracy with respect to optimality. The default value for `opttol` is $1.0e-6$.
- `opttol_abs`: Similarly, `opttol_abs` specifies the final absolute stopping tolerance for the optimality error. A smaller value of `opttol_abs` improves the solution’s accuracy in terms of optimality. The default value for `opttol_abs` is $1.0e-3$.

In order to have a pure absolute stopping test one must set `feastol_abs` $\leq$ `feastol` and `opttol_abs` $\leq$ `opttol`. Initially, setting all these values to $1.0e-6$ was expected to result in Knitro reporting feasibility errors smaller than $1.0e-6$. However, an abnormal behavior was observed where Knitro started reporting feasibility errors of $1e108$ on all data files on some random occasions, which is clearly unreasonable and indicative of a potential bug in the solver itself, rather than a setting issue. This bug was more prevalent on data files where Knitro was violating constraints earlier, however even on files where it didn’t violate constraints started showing this behavior at random times.

- **Knitro solver bug:** Even after setting feasibility options to suitable values, a bug was encountered in Knitro which resulted in Knitro outputting solutions which violated constraints. On closure inspection of solution files it was found that Knitro was able to determine locally optimal solutions but wasn’t reporting them for reason. This bug was particularly prevalent in the `d1` and `d8` data files where this bug was encountered roughly 9 out of 10 times whereas in other files it was encountered hardly 1 out of 10 times. Notably, these 2 are the same files where Knitro was violating constraints before setting the feasibility options to appropriate values. At this point, this behavior was beyond the comprehension of this work and so it was reported to the Knitro team via an email. The corresponding data (`d1` and `d8` as this behavior was most prevalent in these files) and model file along with the options that were used were sent. Options that were being used along with their values are indicated below:

- | | | |
|-----------------------------------|---------------------------------------|--------------------------------------|
| – <code>threads</code> = 4 | – <code>feastol_abs</code> = $1.0e-6$ | – <code>opttol_abs</code> = $1.0e-7$ |
| – <code>feastol</code> = $1.0e-7$ | – <code>opttol</code> = $1.0e-7$ | – <code>ms_enable</code> = 1 |

- ms_maxsolves	=	- ms_maxtime_real	=	- maxit = 10000
100000		300		

What these respective options do can be found at [6]. The initial response by Knitro team was that they are not able to recreate the issue and the problem is being solved on their system. They even sent output log files, which clearly indicated normal behavior when the solver was run on their end. To further assist in diagnosing the issue, log files generated from the Jaltantra system were shared with the Knitro team. They then provided a set of suggestions to explore in an attempt to rectify the observed behavior which are mentioned below:

1. Update AMPL to latest version
2. Update Knitro to latest version
3. Turning off AMPL presolve by setting presolve = 0;
4. Setting number of threads to 1 instead of 4 which would make the solving process slower
5. Terminate on the basis of ms_max_solves rather than ms_maxtime_real

However, options 1, 2, and 3 did not resolve the issue, and the problem persisted with Knitro encountering bugs approximately 9 out of 10 times. Option 4 showed some improvement, with a success rate of 5 out of 10, but it did not completely rectify the issue. Furthermore, using a single thread was not feasible as it could negatively impact the performance of the Jaltantra system. Option 5, while it solved the bug, was not practical to implement due to the variability of the ideal value for ms_max_solves across different problem instances.

These findings were communicated to the Knitro team, who were unable to reproduce the issue on their own system. To gain further insights, the data files were executed on the NEOS server using the same option settings. The bug was observed in the result files generated on the NEOS server as well. The job ID and password were shared with the Knitro team for their investigation.

After a week, the Knitro team informed that they had identified the bug, which occurred specifically when using ms_max_solves as the stopping criterion. They confirmed that the bug had been fixed in the updated binaries, which could be downloaded from the AMPL portal [8]. On replacing the binaries with updated ones, the Knitro bug was finally gone and the solution outputted was adhering to the respective constraints consistently.

4.4 Effect of Computational Parameters on Selected Solver

After preliminary and detailed experimentation, and after result validation, the only two solvers left were Knitro and Baron. After fixing the bug in Knitro, its performance needed to be re-evaluated on computational parameters like threads, CPU time, etc. In order to establish a baseline for comparison, we utilized Baron, the only solver that consistently demonstrated satisfactory performance while adhering to the imposed constraints. However, since we had exhausted all available options and configurations of Baron during the detailed experimentation phase, we concluded that its performance could not be further enhanced. Therefore, Baron's performance was not discussed in this chapter of the report. The best set of settings for Knitro was already identified and discussed during detailed experimentation in Chapter 5 of this report, and they have been mentioned again in Section 6.5.2. These settings were kept constant across all experimentation discussed in this chapter. The performance of Knitro after fixing bugs in section 6.5 with the above-mentioned settings has been highlighted in Table 4.20. It is worth noting that although performance isn't as good as it was before (Table 4.15) but it is still better than Baron. It was now to be seen if the performance could be further enhanced by tweaking the following parameters :

1. Threads
2. Multistart Time
3. Max Iterations

Table 4.20: Knitro Performance with tuned parameters (no bug, 5 minutes)

Data File	Knitro (Obj. Cost in 5 minutes)
d1	4.04E+05
d2	6.06E+06
d3	1.21E+07
d4	1.84E+07
d5	8.82E+06
d6	3.55E+06
d7	3.40E+06
d8	6.92E+06
d9	8.86E+06
d10	9.35E+06
d11	1.07E+07

4.4.1 Effect of Threads

In the previous analysis, as shown in Table 1, it was observed that the performance of Knitro improved with an increasing number of threads. However, this observation

was made during a period when Knitro had a bug and was not adhering to the necessary constraints, as discussed in Section 6.5 of this report. Therefore, it became essential to investigate whether the same performance trend would persist after fixing the bug and ensuring Knitro's adherence to the constraints.

Table 2, referred to as Table 4.21 in this report, presents the objective cost outputted by Knitro within a time frame of 5 minutes for various numbers of threads. The analysis of this table reveals that, for the majority of data files, increasing the number of threads did not result in a significant improvement in the objective cost. However, a minor enhancement was observed for data files d9, d10, and d11. Therefore, if the available computational resources permit, it is recommended to execute Knitro with 4 threads instead of just 1.

Table 4.21: Knitro Objective Costs for different Thread Configurations (5 minutes)

Data File	1 Thread	4 Threads	8 Threads
d1	4.04E+05	4.04E+05	4.04E+05
d2	6.06E+06	6.06E+06	6.06E+06
d3	1.21E+07	1.21E+07	1.21E+07
d4	1.84E+07	1.84E+07	1.84E+07
d5	8.82E+06	8.82E+06	8.82E+06
d6	3.55E+06	3.55E+06	3.55E+06
d7	3.40E+06	3.40E+06	3.40E+06
d8	6.92E+06	6.92E+06	6.92E+06
d9	8.87E+06	8.86E+06	8.85E+06
d10	9.36E+06	9.35E+06	9.32E+06
d11	1.08E+07	1.07E+07	1.07E+07

4.4.2 Effect of Multi-Start Time

Another significant setting of Knitro that had the potential to impact its performance was `ms_maxtime_real`. This parameter determines the maximum time allowed for the multi-start process to run before termination. There are two ways to terminate the solver: either when the specified number of iterations in `ms_maxsolves` is reached or when the time limit specified in `ms_maxtime_real` is reached. However, it is challenging to determine the exact number of multi-start solves needed to achieve a reasonable local optima in advance. Therefore, the time criterion was given more priority.

In the experimentation, `ms_maxtime_real` was set to an extremely high value of 100000, effectively allowing Knitro to perform as many multi-start solves as possible within the given time frame. The rationale behind this choice was that a higher value of

ms_maxtime_real would lead to more solves and, consequently, a higher potential for reaching a better local optima. This hypothesis was tested and results have been shown in Table 4.22. Note that settings were kept constant across different time limits and the number of threads was set to 4. It is observed from Table 4.22 that the differences in

Data File	5 Minutes	1 Hour	4 Hours
d1	4.04E+05	4.04E+05	4.04E+05
d2	6.06E+06	6.06E+06	6.06E+06
d3	1.21E+07	1.21E+07	1.21E+07
d4	1.84E+07	1.84E+07	1.84E+07
d5	8.82E+06	8.82E+06	8.82E+06
d6	3.55E+06	3.55E+06	3.55E+06
d7	3.40E+06	3.40E+06	3.40E+06
d8	6.92E+06	6.92E+06	6.92E+06
d9	8.86E+06	8.85E+06	8.85E+06
d10	9.35E+06	9.32E+06	9.32E+06
d11	1.07E+07	1.07E+07	1.07E+07

Table 4.22: Knitro Objective Costs for Different Time Limits (4 threads)

objective cost between different time frames are not significant. This implies that there is little compromise on the quality of the results when shorter time frames are used. In other words, if there is a need for quick results, a shorter time limit can be set without a substantial impact on the objective cost.

Furthermore, it is evident that even a time limit of 1 hour is more than sufficient to achieve satisfactory results. The marginal improvements obtained by running the solver for longer durations, such as 4 hours, do not justify the additional computational resources and time investment.

4.4.3 Effect of Max Iterations

In addition to the previously discussed settings, Knitro also offers a parameter called maxit, which determines the maximum number of iterations that a single multi-start solve should perform before termination. This setting plays a crucial role in balancing the trade-off between exploring a greater number of starting points versus conducting more iterations to search for a better local minimum.

By setting a small value for maxit, each solve is terminated quickly, enabling the exploration of numerous different starting points. This approach aims to cover a wide range of potential solutions but may not allow sufficient iterations for each solve to converge to a high-quality local minimum.

On the other hand, increasing the value of `maxit` results in a deeper exploration of each solve, allowing the solver to spend more time searching for a better local minimum. This approach focuses on intensifying the search within a smaller number of starting points but may reduce the overall coverage of the solution space.

Both of these experiments are discussed in detail in the section below. The impact of different values of `maxit` on the performance and quality of the results is analyzed and compared. It is important to note that the default value of `maxit` is set to 10,000.

4.4.4 Results

Data File	MAXIT = 50000
d1	4.04×10^5
d2	6.06×10^6
d3	1.21×10^7
d4	1.84×10^7
d5	8.82×10^6
d6	3.55×10^6
d7	3.40×10^6
d8	6.92×10^6
d9	8.85×10^6
d10	9.32×10^6
d11	1.07×10^7

Table 4.23: Increasing `maxit` experiment result

Table 4.23 presents the results obtained from an experiment conducted to investigate the impact of increasing the `maxit` parameter on the objective cost. In this experiment, the `maxit` parameter was set to 50000, five times its default value. The findings of the experiment indicate that increasing the `maxit` parameter did not result in an improvement in the objective cost. This outcome can be attributed to the behavior of the Knitro solver, which terminates a solve instance if it identifies a local optimum, even if the total number of iterations has not reached the specified maximum value. This observation suggests that the `maxit` parameter functions as a maximum limit rather than a directive to enforce a specific number of iterations.

Data File	MAXIT = 2000
d1	4.04×10^5
d2	6.06×10^6
d3	1.21×10^7
d4	1.84×10^7
d5	8.82×10^6
d6	3.55×10^6
d7	3.40×10^6
d8	6.92×10^6
d9	8.85×10^6
d10	9.32×10^6
d11	1.07×10^7

Table 4.24: Decreasing maxit experiment result

Table 4.24 presents the results obtained from an experiment conducted to examine the impact of decreasing the `maxit` parameter on the objective cost. In this experiment, the `maxit` parameter was reduced to 5 times its default value, which amounted to 2000. The experiment revealed that reducing the `maxit` parameter did not lead to any improvement in the objective cost. This outcome aligns with the findings of the previous experiment, where increasing the `maxit` parameter also did not yield better results. Thus it was concluded that `maxit` parameter should be used at its default value.

Chapter 5

New Features

Following the identification of the potential best-fit solver for the Jaltantra model, the focus of this work shifted towards enhancing the features of the current loop Jaltantra system. Various features which could be added in Jaltantra were identified in this work which were then divided among the Jaltantra team to implement. Some of the features which were implemented in this work are discussed in detail in the below sections.

5.1 AMPL solvers integration to Jaltantra

The initial enhancement introduced to the current loop system involved the integration of an additional solver called Knitro, complementing the existing solver Baron. Among the available solver options, Knitro was identified as the most suitable choice for the Jaltantra model after careful evaluation and customization. The integration of Knitro into the system brought several benefits. Firstly, it improved the overall performance by providing multiple solver options. This allowed for greater flexibility in obtaining lower objective values, as each solver might converge to different local optima. Consequently, it increased the chances of finding improved solutions compared to relying on a single solver. Moreover, the inclusion of Knitro addressed the issue of unsolved networks. Some networks that remained unsolved with the single solver approach could now be successfully solved using the combined capabilities of Knitro and Baron. The integration of the solver wasn't straightforward as it had to be done on an already existing code base which demanded the need to document it in this report for future reference.

5.1.1 Methodology

Figure 1.1 shows the architecture of the current loop Jaltantra system. In order to integrate Knitro into the system, following changes had to be done in CalculateNetworkCost.py file:

- In "AutoExecutorSettings" class under the list "AVAILABLE_SOLVERS" string 'knitro' was appended denoting a new solver named Knitro has been added to the system.
- Under the same class in "__update_solver_dict" function, a dictionary named "solvers" exists which stores the path of the solvers, options of the solvers and threads to be used by the solver. Thus a new key named "knitro" (name should be same as used previously) was added and corresponding path of knitro binary file and the optimal settings which were determined earlier in this work were specified as solver option.
- A new class named "SolverOutputAnalyzerKnitro" was created which inherited from the class "SolverOutputAnalyzerParent". In this class 3 functions were defined which are explained below :
 - extract_best_solution : Which accessed the solution files of Knitro and extracted the best cost out of them.
 - check_solution_found : Which determined if Knitro was able to find a solution or not.
 - check_errors : Which determined if Knitro encountered any error or bug. These errors could be license errors, infeasible solution found error, or the bug which was discussed in section 6.5.2 in case it is recreated again. In case any of the above issues is found it would discard the solution file from future consideration and would log the behavior in log files.

5.2 Fixed arcs implementation

An important feature that was present in the branch version of Jaltantra but absent from its loop counterpart was the concept of fixed arcs. Fixed arcs refer to arcs in the network where the pipe diameter is predefined and not subject to determination by the solver. For example, refer to Image 5.1. Suppose, a very large diameter pipe is already available, and it is desired that it used at arc-1. However, if the network is given to the solver, it might use any available diameter. Thus there was a need to tell solver to use only that specific pipe for that arc. This feature is valuable because it allows engineers to incorporate existing pipes into the network and focus on determining the optimal selection of pipes to be used in conjunction with them. By specifying which arcs are fixed, engineers can leverage the solver's capabilities while maintaining control over specific elements of the network

design. To incorporate fixed arcs into the loop Jaltantra system, several modifications were made to the model file and the corresponding structure of the data files. These changes also necessitated adjustments to the methods used for generating data files and extracting results. The modifications to be made mostly revolved around what constraints should be modified which were discussed in section 6.1 of this work. A few approaches were thought of for incorporating fixed arcs into the current loop system some of which are discussed in the below sections.

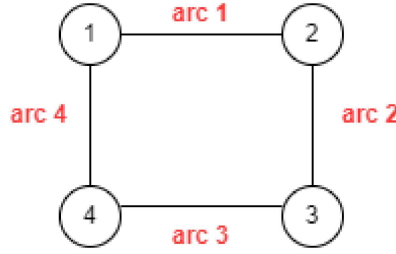


Figure 5.1: Fixed Arc Example

5.2.1 Approach 1

If we look at constraints of Jaltantra model, the only constraint concerned about what length of which pipe should be used for any particular arc is the pipe length constraint which is mentioned in equation (5.1). Another part that involved pipe length is the objective function to be minimized mentioned in equation (5.2). The initial approach revolved around incorporating fixed arcs only through these model components.

$$\forall i, j \in \text{Arcs}, \quad \sum_{\forall k \in \text{Pipes}} l[i, j, k] = L[i, j] \quad (5.1)$$

$$\forall i, j \in \text{Arcs}, \quad \sum_{\forall k \in \text{pipes}} l[i, j, k] \cdot c[k] \quad (5.2)$$

To implement this approach, a new parameter called "fixed_pipe_length[i,j,k]" was introduced in the data file. This parameter, represented as a 3-dimensional array, captured the predetermined length of the kth pipe segment within the (i,j) arc. The purpose of this parameter was to ensure that the kth pipe in the (i,j) arc would have a length equal to or greater than the value specified by "fixed_pipe_length[i,j,k]" in the data file. To enforce this constraint, a new equation, as mentioned in Equation (5.3), was added to the model.

$$\forall i, j \in \text{Arcs}, \quad \forall k \in \text{Pipes}, \quad l[i, j, k] \geq \text{Fixed_pipe_length}[i, j, k] \quad (5.3)$$

Although this approach made sure that fixed arcs will have the pipes which are present in commercially available pipe set, however, problems arose when any arbitrary pipe

diameter had to be used which is not present in the set. This was required due to the k component used in both equations (5.3) and (5.2). If one were to even add arbitrary pipes into the set, a problem would arise as to what must be its cost i.e c[k] component used in equation (5.2). If one sets this cost to 0, then the solver would use this pipe for all arcs as it would set the objective function (5.2) to minimum. Due to the above challenges, this approach was dropped. But this approach did help in acquiring certain intuitions for the right approach which is discussed in the below section. The fixed arcs components shouldn't lead to modification of pipe set as it would interfere with the objective function.

5.2.2 Approach 2

The main idea behind this approach was to treat fixed arcs as a distinct set, separate from the arcs for which the pipe selection problem needed to be solved. By treating them separately, these fixed arcs could be excluded from the objective function, and hence no need to worry about the parameter c[k] i.e. cost. The only thing that required change was constraints which involved arcs as now fixed arcs would also have to be incorporated into them. If we study all the constraints, it is observed that only flow constraint (Equation 5.4) and head loss (Equation 5.5) constraint involved arcs.

$$\forall i \in \text{Nodes}, \quad \sum_{\forall (i,j) \in \text{Arcs}} q[i,j] - \sum_{\forall (j,i) \in \text{Arcs}} q[j,i] = D[i] \quad (5.4)$$

$$\forall i, j \in \text{Arcs}, : \quad H[i] - H[j] = \text{Headloss in } (i,j) \text{ arc} \quad (5.5)$$

For the flow constraint, only a slight modification had to be done such that inflows and outflows also counted for flows from fixed arcs. The modified equation is mentioned in 5.6. For the headloss constraints, an additional constraint had to be added such that the head-loss constraint is also followed in fixed arcs. The additional constraint is mentioned in 5.7.

$$\begin{aligned} \forall i \in \text{Nodes}, \quad & \sum_{\forall (i,j) \in \text{Arcs}} q[i,j] + \sum_{\forall (i,j) \in \text{Fixed Arcs}} F_q[i,j] \\ & - \left(\sum_{\forall (j,i) \in \text{Arcs}} q[j,i] + \sum_{\forall (j,i) \in \text{Fixed Arcs}} F_q[j,i] \right) = D[i] \end{aligned} \quad (5.6)$$

$$\forall i, j \in \text{Fixed arcs}, : \quad H[i] - H[j] = \text{Headloss in } (i,j) \text{ Fixed arc} \quad (5.7)$$

In the data file corresponding adjustments had to be made to incorporate the fixed arc set and its properties. A new set named "F_arcs" was added. The properties of fixed arc like arc length, what diameter pipe to use in this arc and what is the roughness of used pipe was added in terms of parameter named "F_L", "F_d", and "F_R" respectively. A separate flow variable named "F_q[i,j]" was added which denoted flows in fixed arc.

5.2.3 Verification of fixed arc

In order to validate the results of the fixed arc implementation, a three-step test was devised. The test involved the following steps:

- Step 1: The first step involved solving a data file without any fixed arcs and analyzing the resulting solution file to determine which pipes were used for each segment.
- Step 2: In the second step, a subset of arcs from the solution file obtained in Step 1 was selected. The network was then resolved with the same data, but this time with the selected arcs designated as fixed arcs. The fixed arcs retained the diameters assigned to them in Step 1.
- Step 3: The solutions obtained in Step 1 and Step 2 were compared. If the solutions were found to be similar, it would provide a strong indication that the implementation of fixed arcs was correct.

This test was performed on a very small data file with 8 nodes and 2 loops. The solution obtained for different arcs is shown in Table 5.1. Now for step 2, Arc 1 and Arc 2 were chosen to be fixed arcs with diameters 457.20 and 406.4 respectively. The solution obtained in step 2 is shown in table 5.2.

Table 5.1: Solution obtained in Step 1

Arc	Start Node	End Node	Length	Flow	Diameter
1	1	2	1000	311.11	457.2
3	2	3	793.106	102.31	254
3	2	3	206.894	102.31	304.8
2	2	4	1000	181.02	406.4
4	3	5	97.8522	74.54	203.2
4	3	5	902.1478	74.54	254
5	4	5	1000	0.27	25.4
6	4	6	304.861	147.41	355.6
6	4	6	695.139	147.41	406.4
8	6	7	10.6054	55.75	203.2
8	6	7	989.3946	55.75	254
7	7	5	1000	0.19	25.4

The comparison of the solutions obtained in Step 1 and Step 2 reveals that they are identical in terms of diameter, flow, and length. This observation strongly indicates that the implementation of fixed arcs is correct. Furthermore, similar tests were conducted on other data files with randomly selected fixed arcs, and consistent results were obtained

Table 5.2: Solution obtained in Step 2

Pipe ID	Start Node	End Node	Length	Flow	Diameter
1	1	2	1000	311.11	457.2
3	2	3	793.068	102.31	254
3	2	3	206.932	102.31	304.8
2	2	4	1000	181.02	406.4
4	3	5	97.8521	74.54	203.2
4	3	5	902.1479	74.54	254
5	4	5	1000	0.27	25.4
6	4	6	304.755	147.41	355.6
6	4	6	695.245	147.41	406.4
8	6	7	10.6052	55.75	203.2
8	6	7	989.3948	55.75	254
7	7	5	1000	0.19	25.4

across all the tests. These findings provide further validation of the accuracy and reliability of the fixed arc implementation in the Jaltantra system.

5.3 Database for Storing Solver Result

Current loop Jaltantra system used file system to store record of the solves that were being done. This made it tedious to analyse how the solvers were performing for different data files. The unique hash-id of each solve had to be fetched and then manually the records of that particular file had to be analysed. In order to gain better insights into the performance of different solvers on various water networks within the Jaltantra system, a dedicated database was implemented as an additional feature. The purpose of this database was to store the results obtained by the solvers for different data files using the available model files at one place and to fetch them conveniently. This feature was incorporated with the intention of facilitating future decision-making regarding the procurement of commercial licenses for the Jaltantra system. The architecture and working of the database have been discussed in detail in the below sections.

5.3.1 Database architecture

The implementation of the dedicated database, named "networks.db," involved careful planning and consideration of the tables and their respective schemas that would be included in the database. This planning phase involved defining the structure and organization of the database, ensuring that it effectively captured and stored the relevant information related to solver results for different water networks. It was decided that

database would have two tables namely all_solves and best_solves. Figure 5.2 shows the architecture of "networks.db" database.

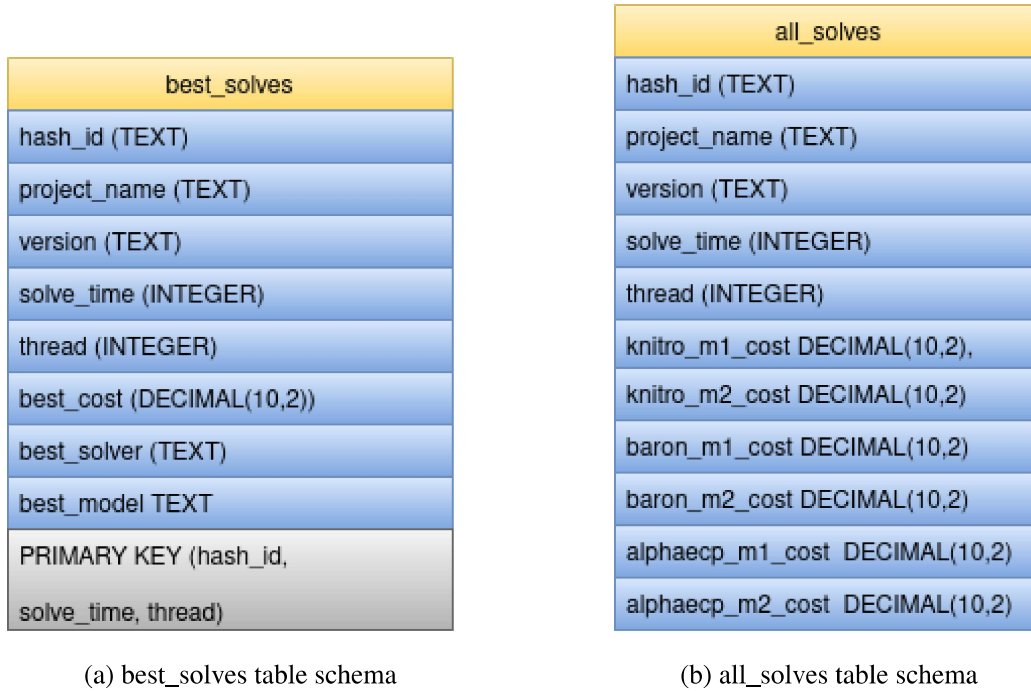


Figure 5.2: networks.db architecture

- **best_solves** : This table would only store data about the best solution that was generated for the data files or water networks. This table plays a crucial role in determining which solvers and models consistently produce the best results, aiding in the decision-making process for selecting solvers and models for the future. Figure 5.2a shows the schema of the best_solves tables. The detailed description of the schema is given below :

- Hash_id : Represents the unique identifier or hash value associated with the network. This field serves as part of the primary key and helps to uniquely identify each solver session.
- Project_name : Denotes the name or identifier of the project or water network. Since hash_id is too long and generation of hash_id is tedious, this field makes querying into the database easier when information related to any particular project is required.
- Version : Defines the version number of the loop Jaltantra system that gave the best result. This field can help in determining how much the performance of system evolved in terms of objective cost across different versions.

- solve_time : Specifies how much time did the solver take to solve the network file to produce the best result.
- threads : Specifies how many threads did the solver which produced the best result utilized.
- best_cost : Stores the objective cost or value obtained from the best solution for the data file / water network.
- best_solver : Specifies the name or identifier of the solver that produced the best solution.
- best_model : Represents the name or identifier of the model file used for producing the best solution.

Hash_id of the network, time used and number of threads involved were used as the primary key as these three parameters will always be unique to a solver session. If a solver session with the same hash_id, solve_time, and threads parameters is encountered, the new result will replace the existing one. Otherwise, the new result will be appended to the table.

- **all_solves** : This table would store record of all the solves by all the solvers that have been done by the loop Jaltantra system. This would aid in the comparison of solver and model performance on different data files / water networks. Figure 5.2b shows the schema of the all_solves table. Hash_id, project_name, version, solve_time, threads are same fields as discussed in best_solves table hence they have been excluded in this discussion. New fields added to all_solves table are discussed below :

- knitro_m1_cost : Denotes the objective cost reported by knitro solver using m1 model file for the water network. In case no solution or infeasible solution is found by knitro using m1 model, this field would be filled as infinity. This holds true for all the fields discussed below as well.
- knitro_m2_cost : Denotes the objective cost reported by knitro solver using m2 model file.
- baron_m1_cost : Denotes the objective cost reported by baron solver using m1 model file.
- baron_m2_cost : Denotes the objective cost reported by baron solver using m2 model file.

- `alphaecp_m1_cost` : Denotes the objective cost reported by alphaecp solver using m1 model file.
- `alphaecp_m2_cost` : Denotes the objective cost reported by alphaecp solver using m2 model file.

Since this table is for recording all the solutions that have been done by the loop system, no primary key was constructed. A new record would be appended to the table even if a previous record with exact same specification existed.

After finalizing the table schemas, the next step was to design the integration of the database into the existing Jaltantra system. This phase of the work focused on determining how the system would interact with the database, including the processes for storing, retrieving, and updating solver results. The architecture of database integration is shown in Figure 5.3. The database used is `sqlite3` and has been integrated via the following functions in `CalculateNetworkCost.py` :

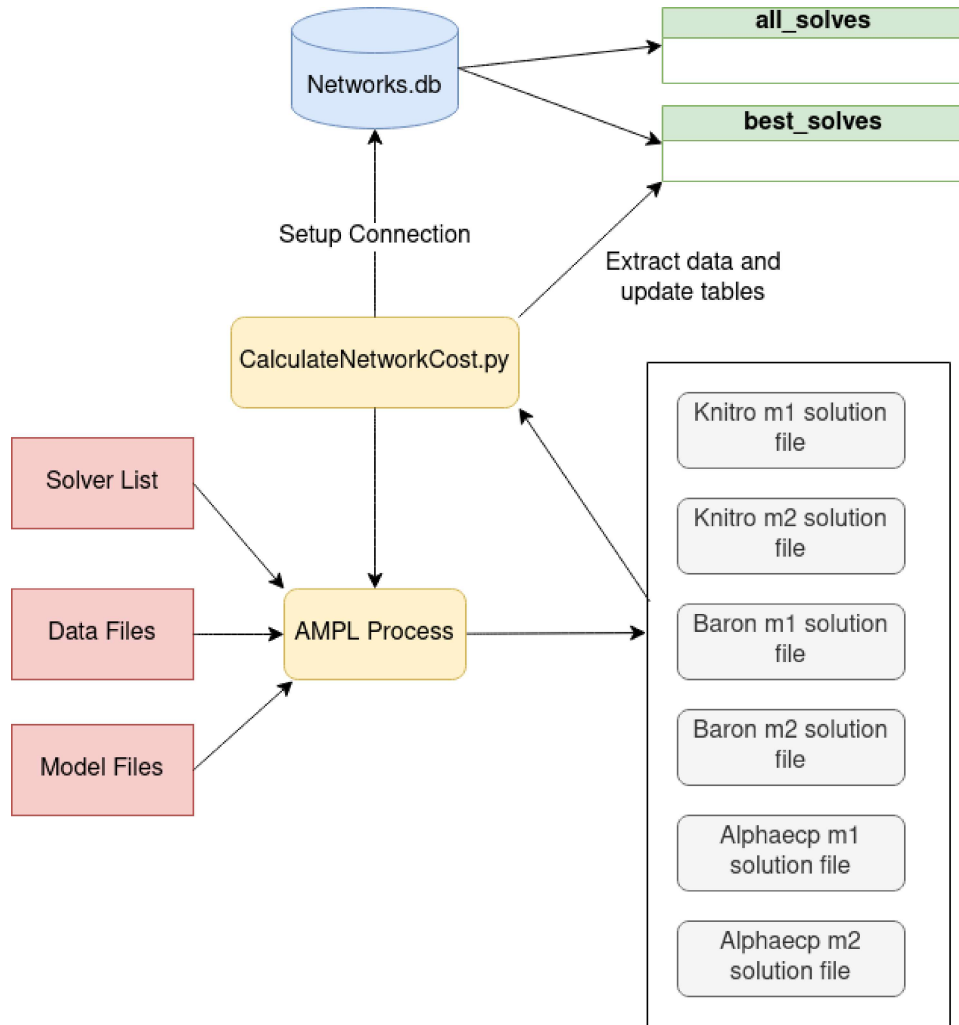


Figure 5.3: Database integration architecture

- **setup_database()** : This function is called in the main() function of the program and establishes the connection and cursor for interacting with the database. It also checks if tables all_solves and best_solves exist in the database, and if not it runs sql queries to create these tables with the schema discussed above. This is the function where changes have to be made in case the schema of the table is to be changed or new tables have to be added.
- **update_all_solves_tables()**: This function is called in the extract_best_solution() function which accesses all the solution files to determine the best one. While it does that, all the objective costs are sent to this function along with the solver name

and model file to this function which then runs an sql query to update the all_solves table.

- **update_best_solves_table():** This function is called in main_write_final_solution function of the program which writes information about the best solve in a file. After it is done with the file aspect, this function calls the update_best_solves_table() function with information about the best solver, model, and cost as arguments. In this function, all these records are updated in the table via an sql query. Finally, the connection to the database is also closed here.

5.4 Other contributions

Apart from the substantial work carried out in the areas of solver filtering, tuning, and system enhancements, this research also encompassed minor contributions in other facets. These contributions are outlined below:

- **Documentation of Back-End Files:** A comprehensive documentation effort was undertaken for the back-end files utilized by the system. This documentation encompassed Python files responsible for spawning AMPL sessions and extracting results, as well as shell scripts that invoked these Python files. Additionally, thorough comments were added to the AMPL model files to enhance their comprehensibility. This detailed documentation serves as a valuable resource, enabling individuals to acquaint themselves with the codebase.
- **Setting up of GitHub:** An official GitHub repository was established under the Jaltantra organization to host the codebase of the loop Jaltantra system. This ensures proper version control, collaboration, and code sharing among team members and potential contributors.
- **Preparation of Installation Guide:** A comprehensive and detailed installation guide was meticulously created for the various components of the loop Jaltantra system. This guide serves as a valuable resource for anyone involved in future development within the project, providing them with step-by-step instructions and guidance on setting up the system.
- **Resolution of tomcat issues :** One of the primary challenges encountered was ensuring the automatic startup of Tomcat on system restart, eliminating the need for manual intervention. By configuring the necessary settings, Tomcat was configured to automatically initiate upon system startup. After updating Tomcat to version

10 system stopped working, and after conducting a thorough investigation, it was determined that the issue stemmed from the fact that Tomcat 10 had discontinued support for the "javax" namespace, which was extensively utilized in our project.

Chapter 6

Results

After all the experimentations were done in this work, Knitro and Baron were filtered as the best solvers for Jaltantra loop system. After parameter tuning, the best settings for operating Baron and Knitro are given below :

- Knitro :

- | | |
|------------------------|-------------------------|
| – threads = 4 | – ms_enable = 1 |
| – feastol = 1.0e-7 | – ms_maxsolves = 100000 |
| – feastol_abs = 1.0e-6 | – ms_maxtime_real = 300 |
| – opttol = 1.0e-7 | – maxit = 10000 |
| – opttol_abs = 1.0e-7 | |

These settings made sure that Knitro utilized as many starting points as possible and did solves from them in a time period of 5 minutes. Feasibility setting ensured that results obtained were consistently adhering to the constraints.

- Baron :

- | | |
|-----------------|------------------------------|
| – threads = 1 | – nlpsol = Default (Use all) |
| – maxtime = 300 | – lpsolver = Default |

The performance of Knitro and Baron with these settings is compared in Table 6.1. It is observed that Knitro always gives the better or same result as Baron except for data file d10 where Baron is marginally better. However, if threads of Knitro are increased to 8, it is observed Knitro is also able to perform the same as Baron for d10 data file as well. On the other hand, Baron has given unsatisfactory results (objective cost 10 times higher than Knitro) for 3 data files (d5, d7, d9). When Baron team was contacted to inquire if it's possible to make Baron work better on these files, they replied stating that Knitro with multi-start is a better choice for these problems. This further affirms that this work has been successful in determining the best potential solver for loop Jaltantra.

Table 6.1: Comparison of Objective Costs between Baron and Knitro Solvers at Optimal Settings (5 minutes)

Data File	Baron Objective Cost	Knitro Objective Cost
d1	4.04E+05	4.04E+05
d2	6.06E+06	6.06E+06
d3	1.21E+07	1.21E+07
d4	1.84E+07	1.84E+07
d5	No acceptable solution	8.82E+06
d6	3.55E+06	3.55E+06
d7	No acceptable solution	3.40E+06
d8	6.92E+06	6.92E+06
d9	No acceptable solution	8.86E+06
d10	9.32E+06	9.35E+06
d11	1.07E+07	1.07E+07

While this work focuses on AMPL solvers, my peer worked on GAMS related solvers. That work concluded with Alphaecp as the best solver. Table 6.2 shows the comparison between these Alphaecp and Knitro. One thing to note here is that, AlphaECP can use either Ipopt or Conopt as subsolver. While the license of Ipopt is free, Conopt is paid. The performance of Alphaecp with both these sub-solvers is still slightly worse than Knitro. The affirms that Knitro is potentially the best solver for the loop Jaltantra system.

Table 6.2: Comparison of Objective Costs between AlphaECP and Knitro (5 minutes)

Data File	AlphaEcp (Conopt)	AlphaEcp (Ipopt)	Knitro
d1	4.04E+05	4.17E+05	4.04E+05
d2	6.07E+06	6.11E+06	6.06E+06
d3	1.21E+07	1.22E+07	1.21E+07
d4	1.84E+07	1.86E+07	1.84E+07
d5	8.84E+06	8.90E+06	8.82E+06
d6	3.56E+06	3.60E+06	3.55E+06
d7	3.41E+06	3.40E+06	3.40E+06
d8	6.93E+06	6.92E+06	6.92E+06
d9	9.13E+06	8.92E+06	8.86E+06
d10	9.75E+06	9.37E+06	9.35E+06
d11	1.08E+07	1.15E+07	1.07E+07

Another aspect of this work was to analyze the performance of different model files with solvers. While the m1 model file always gave results with Knitro, m2 on the other hand wasn't as consistent indicating that m1 might be the better choice. Table 6.3 shows how Baron and Knitro performed with m1 and m2 model files on optimal settings and

with a time frame of 5 minutes. Table 6.4 shows how best results obtained from m1 and m2 compare. Note that a negative gap indicates m1 performed better and positive would indicate m2 performed better. However it must be noted that while using the loop Jaltantra system, it was observed that even m2 model gave better results for some water networks and hence it is not completely justified to exclude it from the system.

Table 6.3: Comparison of models (5 Minute solve time)

Data File	Baron (m1)	Baron (m2)	Knitro (m1)	Knitro (m2)
d1	4.04E+05	4.04E+05	4.04E+05	4.04E+05
d2	6.06E+06	6.06E+06	6.06E+06	6.06E+06
d3	1.21E+07	1.25E+07	1.21E+07	1.21E+07
d4	1.84E+07	1.92E+07	1.84E+07	1.86E+07
d5	No acceptable solution	8.82E+06	8.82E+06	8.82E+06
d6	3.55E+06	4.25E+06	3.55E+06	3.56E+06
d7	No acceptable solution	No acceptable solution	3.40E+06	3.40E+06
d8	6.92E+06	6.96E+06	6.92E+06	6.92E+06
d9	No acceptable solution	No acceptable solution	8.86E+06	Infeasible Solution
d10	9.32E+06	No acceptable solution	9.35E+06	Infeasible Solution
d11	1.07E+07	No acceptable solution	1.07E+07	Infeasible Solution

Table 6.4: Comparison of Best Results of model files

Data	M1 Model File	M2 Model File	Gap (%)
d1	4.04E+05	4.04E+05	0.00%
d2	6.06E+06	6.06E+06	0.00%
d3	1.21E+07	1.21E+07	0.00%
d4	1.84E+07	1.84E+07	0.00%
d5	8.82E+06	8.82E+06	0.00%
d6	3.55E+06	3.56E+06	-0.28%
d7	3.40E+06	3.40E+06	0.00%
d8	6.92E+06	6.92E+06	0.00%
d9	8.85E+06	8.92E+06	-0.78%
d10	9.32E+06	1.07E+07	-12.90%
d11	1.07E+07	1.10E+07	-2.73%

Table 6.5 shows under what scenario the best result was obtained for particular data files. It is clear from table 6.5 that for every data file, the best case scenario always involved Knitro hence reaffirming that knitro is potentially the best fit as a solver for loop Jaltantra system.

Table 6.5: Scenario for best result

Data	Best result scenario (5 MINUTES CPU TIME)
d1	Knitro & Baron gave the best result, which is the same across all threads. AlphaEcp with Conopt with m1 also gave similar results. However, for m2 and Ipopt, AlphaEcp didn't perform well.
d2	Same situation as above.
d3	Knitro and AlphaEcp (Conopt) gave the same best result across all combinations for all models. Baron gave the same results as Knitro and AlphaEcp for m1 but performed worse for m2. AlphaEcp with Ipopt performed the worst.
d4	For m1, all solvers gave the same best result except for AlphaEcp with Ipopt. For m2, only Knitro with 8 threads gave the same best result.
d5	Knitro gave the best results across all threads. Baron also gave the best results for m2 and didn't solve for m1 at all.
d6	Knitro and Baron gave the best results for m1 across all threads.
d7	Knitro and AlphaEcp (both) gave the best results for the m1 model across all threads.
d8	All three solvers gave the same best result for m1 across all threads. For m2, only Knitro with 4 and 8 threads gave this result.
d9	The best result was obtained for Knitro with 8 threads for the m1 model. There was a 0.2% difference in the result between 1 thread and 8 threads. No solver was able to solve the m2 model, and Baron was not able to give proper results for both models.
d10	The best result was obtained for Knitro with 8 threads. No solver was able to solve the m2 model.
d11	The best result was obtained for Baron for the m1 model. Using 4 threads, Knitro and Baron gave the best result.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

In the initial phases of this work, NEOS was thoroughly explored to understand its working and see its relevance in finding the right solver for JalTantra. All its interfaces were explored in hopes of finding something which could be automated with the JalTantra model and data files. It was found out that automation is not possible as all automatable options are restricted in terms of the size of the data file i.e. number of variables and constraints that are generated.

Out of 60 plus solvers available on NEOS, Knitro was filtered out on the basis of its performance after preliminary and detailed experiments. It was later tuned to find the best set of parameters and options so that it performs best for the Jaltantra model. Table 6.1 and 6.2 provide a comparative analysis between Knitro, Baron, and Alphaecp clearly demonstrating the superior performance of Knitro. The findings highlight the effectiveness of the tuned parameters and options, reinforcing the suitability of Knitro for the Jaltantra model. A similar attempt was made with the Baron solver. However, despite comprehensive efforts, no specific set of parameters and options could be discovered for Baron to enhance its performance for the Jaltantra model.

A constraint checker was built to check if solutions given by the solver adhered to the constraints, and later this checker was integrated with the Jaltantra system. It led to the discovery that both Baron and Knitro were violating constraints due to issues in settings, some integration bugs, and also bugs in the solver. All of these issues and bugs were solved so that the solvers always adhered to the constraints.

Following the identification of the optimal solver, the focus of this study shifted towards augmenting the capabilities of the loop Jaltantra system. This involved implementing enhancements aimed at improving its functionality and performance. One

notable enhancement was the addition of support for fixed arcs within the system. This feature empowers users to designate a predetermined set of pipes for specific arcs, providing greater control and flexibility in the modeling process.

Furthermore, to facilitate comprehensive analysis and evaluate solver and model performance, a dedicated database was integrated into the system. This database systematically records all the solutions generated by the system, serving as a valuable resource for future analysis and decision-making. Specifically, it assists in making informed choices regarding the purchase of commercial licenses for specific solvers and determining which models are best suited.

Overall, this work has made significant contributions towards enhancing the system's overall performance both in terms of objective cost and features available to the user. By combining the power of an optimal solver with additional features such as support for fixed arcs and the inclusion of a comprehensive database, the system has been made better in terms of performance and functionality.

7.2 Future work

The current work has laid a solid foundation for the loop Jaltantra system by incorporating multiple solvers and implementing a dedicated database for performance analysis. Moving forward, there are several areas that can be explored to further enhance the system's capabilities. The following future work is recommended:

- **Generation of Test Cases and Data Files:** To identify the most suitable solver among the three currently being used, it is essential to conduct comprehensive testing using various test cases and data files. These test cases should cover a diverse range of network sizes, including both small and large networks. Additionally, the test cases should encompass scenarios with fixed arcs as well as those without fixed arcs.
- **Evaluation and Selection of Commercial License:** The current system is using 3 solvers namely Knitro, Baron, and Alphaecp with academic licenses. After using the system on a lot of data files and networks, a thorough analysis has to be carried out to determine which solver's commercial license should be bought before releasing it for public use.

- **Maximum / Minimum allowable Flow Speed:** The original branch Jaltantra system had an additional feature of specifying the maximum and minimum allowable flow speed through pipes. A similar feature can be added to loop version as well. This would require adding additional constraints to the model however caution has to be made that these constraints don't worsen the performance of the system.
- **Dynamic Determination of Threads and Solve Time:** A module can be developed to dynamically determine the optimal number of threads and solve time for solving water networks in the loop Jaltantra system. Currently, the system uses a fixed number of threads and solve time, regardless of the network's characteristics. This can result in inefficient resource allocation and unnecessary waiting time for users. By incorporating a module that considers factors such as network size, the presence of fixed arcs, and the number of arcs in the water network, the system can adapt its resource allocation to each specific case. This would allow for faster solving times for small networks and better utilization of available threads for larger networks.

References

- [1] Hooda, Nikhil, and Om Damani. "JalTantra: A system for the design and optimization of rural piped water networks." *INFORMS Journal on Applied Analytics* 49, no. 6 (2019): 447-458.
- [2] Hooda, Nikhil, Ashutosh Mahajan, and Om Damani. "A Series of ILP models for the optimization of water distribution networks." *Sāadhanā* 44, no. 12 (2019): 1-19.
- [3] <https://neos-server.org/neos/>
- [4] <https://www.epa.gov/water-research/epanet>
- [5] <https://neos-guide.org/users-guide/neos-interfaces/>
- [6] https://www.artelys.com/docs/knitro//2_userGuide/algorithms.html
- [7] https://en.wikipedia.org/wiki/Hazen%E2%80%93Williams_equation#:~:text=The%20Hazen%E2%80%93Williams%20equation%20is,supply%20networks%2C%20and%20irrigation%20systems.
- [8] <https://portal.ampl.com/account/ampl/login>
- [9] <https://dev.ampl.com/solvers/baron/options.html>
- [10] <https://octeract.gg/octeract-engine/>
- [11] <https://docs.octeract.com/>

Acknowledgement

I would like to express my deep sense of gratitude to **Prof. Om Damani**, Department of Computer Science and Engineering, Indian Institute of Technology Bombay, for his generous guidance on how to carry out this research work and for always showing the right direction in case of difficulty. I would also like to thank **Prof. Ashutosh Mahajan** for giving guidance regarding solvers and **Nikhil Hooda** for suggesting approaches for implementing fixed arc feature.

Also, I would like to thank **Prof. Varsha Apte**, Department of Computer Science and Engineering, Indian Institute of Technology Bombay, for teaching me how to properly format and present my ideas through a report. Lastly, I am extremely thankful to my classmate **Manal Jain** for giving his useful suggestions at various steps during this work.

Siddhant Singh

IIT Bombay

-